STUDY GUIDE

PikaByte

Hardware, Networks & Behaviorist AI



Version 1.0 | November 2026

Table of Contents

Table of Contents 2
Hardware and data 4
Chapter 1: Number Systems 5
Stage 1: Computers and Data 5
Stage 2: Number Systems 20
Stage 3: Data Storage 29
Chapter 1.1 Summary 39
Chapter 2: Hardware and the Von Neumann Model 44
Stage 1: Von Neumann Model 44
Stage 2: Computer Hardware 50
Stage 3: Motherboard Power and BIOS, CPU, and GPU 63
Stage 4: Graphics Memory, RAM, ROM, and Storage 70
Networks & Security 80
Chapter 3: Network and Internet Basics 81
Stage 1: What Is a Computer Network? 81
Stage 2: Network Fundamentals 89
Chapter 4: Network Security 101
Stage 1: Security in Networks 101
Behaviorist AI 112
Chapter 5: Intro to Behaviorist AI & RL 113
1. What is Behaviorist AI? 113
2. The Core Process: Reinforcement Learning 114
3. Model-Based vs. Model-Free Learning 115
Summary & Glossary 116
Chapter 6: RL Algorithms and MDPs 118
Background: AI and Machine Learning 118
Stage 1: Markov Decision Processes (MDPs) 120
Stage 2: RL Algorithms and Techniques 125
Chapter 7: AI in Robotics and Games 131
Stage 1: Core Theoretical Foundations 131

Stage 2: Applications and Use Cases of Behaviorist AI 132
Stage 3: Chapter Exercise 134
Chapter 8: AI Challenges and Limitations 137
1 Challenges and Limitations 137
Summary 144



Hardware and data

Learning Goals

- 1) Define a computer and explain its basic workflow (Input, Processing, Storage, Output).
- 2) Convert between binary, decimal, and hexadecimal number systems.
- 3) Explain how different data types (numbers, text, images, audio, video) are stored as binary data.
- 4) Distinguish between primary storage (RAM) and secondary storage (SSD/HDD).
- 5) Describe the Von Neumann model and how internal hardware components (CPU, ALU, Memory) work together to execute a program.
- 6) Identify core hardware components and distinguish between internal hardware and external peripherals.
- 7) Define a computer network and distinguish between different network types (LAN vs. WAN).
- 8) Explain how the internet delivers data using TCP/IP, IP addresses, and domain names (DNS).
- 9) Identify the four core network security goals (Privacy, Authentication, Integrity, Nonrepudiation) and match common protections to each. Color

Chapter 1: Number Systems

Stage 1: Computers and Data

LEARNING GOALS

By the end of this stage, you will be able to:

1. Identify the core physical components of computers and explain their basic functions.

TOPICS COVERED

1.1 1.1 Definition of a Computer
1.2 1.2 Turing Model
1.3 1.3 von Neumann Model
1.4 1.4 Computer Components
1.5 1.5 Workflow of a Computer: Input
1.6 1.6 Workflow of a Computer: Data Processing
1.7 1.7 Workflow of a Computer: Output
1.8 1.8 Workflow of a Computer: Storage
1.9 1.9 Different Types of Computers

1.1 Definition of a Computer

A **computer** is an electronic machine that takes in **data**, works on it by following a list of **instructions**, and gives back a result.

Let's define the words we just used:

- **Data**: Raw information a computer can work with, such as numbers, text, pictures, or sound.
- **Instruction**: A single step that tells the computer what to do (for example, "add these two numbers").
- **Program**: A complete list of instructions that tells the computer how to finish a task.

Every computer does four basic jobs:

#	Function	What it means	Example
1	**Input**	Receives data from the outside world	Typing on a keyboard, moving a mouse, a sensor reading
2	**Processing**	Works on the data using the program's instructions	Adding numbers, comparing two values
3	**Output**	Sends the finished result back to the user	Showing text on a screen, printing a page
4	**Storage**	Saves data so it can be used later	Saving a file to a hard drive

Computers come in many forms:

- A **personal computer (PC)** is a computer made for one person to use, like a desktop or laptop.
- A **smartphone** is a pocket-sized computer that also works as a phone.
- A **supercomputer** is an extremely powerful computer used for huge tasks such as scientific research.

WORKED EXAMPLE

You use a calculator app on your phone to add 5 + 3.

Step	What happens	Function
1	You tap 5, +, 3 on the touchscreen	Input
2	The phone adds the numbers and gets 8	Processing
3	The phone shows 8 on the screen	Output
4	The phone keeps the answer in its history list	Storage

ACTIVITY

You take a photo with your phone and save it to the gallery. Which of the four basic functions is happening in each step?

a. The camera captures the picture.
b. The phone turns the picture into a file it can handle.
c. The picture appears on your screen as a preview.
d. The photo is kept in the gallery.

ANSWER

a. **Input**: the camera gathers data from the outside world.
b. **Processing**: the phone works on the data.
c. **Output**: the result is shown to the user.
d. **Storage**: the data is saved for later use.

1.2 Turing Model

In 1936, a mathematician named **Alan Turing** proposed the idea of a machine that could do *any* calculation, as long as it was given the right instructions. This idea is called the **Turing Machine**. A **Turing Machine** is an imaginary (hypothetical) device with three parts:

- A very long tape (it never ends) where symbols are written and read.
- A head that reads and writes symbols on the tape.
- A set of **rules** that tell the head what to do next, based on the symbol it sees and its current state.

[insert image about a Turing Machine with its tape and head]

From this idea comes the **Turing model**, which describes a computer as a **programmable data processor**. *Programmable* means you can change what the computer does by changing its program. The computer is treated like a **black box**: a device where you only see what goes in and what comes out, not what happens inside.

[insert image about a black box diagram: Input data -> Computer -> Output data]

In this model, the output depends on **two things**: the **input data** and the **program**. Changing either one can change the result. But running the *same program* with the *same input* always gives the *same output*. Turing's work also showed that any problem that *can* be solved by following steps can be solved by a computer, as long as it has the correct program.

WATCH OUT

This model is a little too broad. Even a simple calculator fits this description, but we don't usually call a calculator a full computer.

WORKED EXAMPLE

The input data is always 3, 12, 8, 22. Watch how the output changes when only the program changes.

Program	Input data	Output data
Sort (put in order, smallest first)	3, 12, 8, 22	3, 8, 12, 22
Add (find the total)	3, 12, 8, 22	45
Find smallest	3, 12, 8, 22	3
Find smallest	3, 12, 8, 22	3

[insert image about one computer running three different programs (Sort, Add, Find smallest) on the same input]

> **ACTIVITY**
>
> Your input data is `5, 2, 9`. What is the output for each program?
>
> a. Sort
> b. Add
> c. Find smallest

ANSWER

a. **Sort** gives `2, 5, 9`
b. **Add** gives `16` (5 + 2 + 9)
c. **Find smallest** gives `2`

The input stayed the same, but the program changed the output.

1.3 von Neumann Model

Around 1944–1945, a mathematician named **John von Neumann** added a new idea to Turing's work. He pointed out that **programs and data are both just information**, so a computer can keep *both* in its **memory**.

Memory is the part of the computer that holds programs and data while the computer is working. Before this idea, early computers had to be rewired by hand every time they needed to do a new job. With the von Neumann model, switching tasks is as easy as loading a different program. This model is the base of most modern computers.

The **von Neumann model** splits a computer into four parts, called **subsystems**:

Subsystem	What it does
Memory	Stores both programs and data
Arithmetic Logic Unit (ALU)	Does the math (like adding) and the logic (like comparing two numbers)
Control Unit	Manages and directs all the other parts, like a traffic controller
Input/Output	Moves data between the computer and the outside world

The control unit repeats a simple cycle: it **fetches** (gets) an instruction from memory, then **executes** (carries out) it, then moves on to the next one, one after another. This keeps programs running in an orderly and predictable way.

[insert image about the von Neumann model showing Memory, ALU, Control Unit, and Input/Output]

WORKED EXAMPLE

Add two numbers, 4 and 6.

Step	Subsystem	What it does
1	Input/Output	Receives the numbers 4 and 6 from the keyboard
2	Memory	Stores 4 and 6, along with the "add" instruction
3	Control Unit	Fetches the "add" instruction from memory
4	ALU	Calculates 4 + 6 = 10
5	Input/Output	Sends 10 to the screen

ACTIVITY

Match each job to the correct subsystem: **Memory**, **ALU**, **Control Unit**, or **Input/Output**.

a. Checking whether 15 is bigger than 12
b. Holding a program while it runs
c. Showing a result on the monitor
d. Deciding which instruction runs next

ANSWER

a. **ALU** (comparing is a logic operation)
b. **Memory**
c. **Input/Output**
d. **Control Unit**

1.4 Computer Components

A computer is made of three main components:

Hardware: The physical parts of the computer that you can touch.

Software: The programs and instructions stored in the computer's memory.

Data: Information the computer can read, change, or store.

Hardware includes memory, the ALU, and the input/output parts. **Input** means the data or commands the computer receives from users or devices. **Output** means the result the computer gives back (such as a display or a printout). Together, they let the user and the computer communicate.

[insert image about an input example (keyboard to screen) and an output example (screen to printer)]

Computers store and process data in **binary**, a number system that uses only two digits: `0` and `1`. Numbers, text, pictures, and sound must all be converted into binary before a computer can work with them. **Software** is what lets the hardware and the data work together, because it tells the hardware how to handle the data. The idea of software began with early computers, where programs were set up by hand.

Software has four key parts:

1. **Stored programs**: Modern computers keep both programs and data in memory. Early computers needed to be set up by hand for each task.
   [insert image about memory split into a Program section and a Data section]
2. **Sequence of instructions**: A program is a series of instructions that work on data. The same instructions can be reused and rearranged to build different programs.
   [insert image about a four-step instruction list: input first number, input second number, add and store, output result]
3. **Algorithms**: An **algorithm** is a step-by-step set of rules for solving a problem. Programmers plan an algorithm before they write a program.
   [insert image about an algorithm flowchart: Input -> Set of rules -> Output]
4. **Languages**: Early computers only understood **machine language**, which is written entirely in binary.
   Later, **symbolic languages** were created, which use letters and symbols so people can write programs more easily. The first standard one is **ASCII** (explained in section 1.2.2), which gives every character its own number and binary code:

Character	ASCII code	Binary code
null character	0	0000000
a	97	1100001
b	98	1100010
c	99	1100011
A	65	1000001
B	66	1000010
C	67	1000011
%	37	0100101
+	43	0101011
0	48	0110000
1	49	0110001
Delete	127	1111111

WORKED EXAMPLE

Describe the sequence of instructions for adding two numbers.

1. Input the first number into memory.
2. Input the second number into memory.
3. Add the two numbers and store the result in memory.
4. Output the result.

These steps are also a small **algorithm**: a clear, step-by-step way to solve the problem of adding two numbers.

> **ACTIVITY**
>
> Write a short algorithm (3 to 5 steps) for finding the **average** of two numbers. (The average is the total of the numbers divided by how many numbers there are.)

ANSWER

One possible answer:

1. Input the first number into memory.
2. Input the second number into memory.
3. Add the two numbers together.
4. Divide the total by 2 and store the result in memory.
5. Output the result.

Any answer with clear, ordered steps that gives the right result is correct.

1.5 Workflow of a Computer: Input

A **workflow** is the path that data follows from start to finish. In a computer, data moves through four parts:

Input → Processing → Output, with **Storage** connected to the processing step so data can be saved and retrieved.

[insert image about the computer workflow: input, processing, output, with storage connected to processing]

For a computer to produce any **output**, it first has to receive **input** to work on. An **input system** has four elements:

1. **Input interface**: The part that collects data from outside and delivers it to the computer. It includes:
    - **Human Interface Devices (HID)**: Devices people use to talk to the computer, such as keyboards, mice, and touchscreens.
    - **Input/Output ports**: Physical connection points, such as USB ports, audio jacks, and HDMI ports, where devices plug in.
2. **Input controllers**: Small circuits that receive signals from input devices, prepare the data, and pass it to the **central processing unit (CPU)**. (The CPU is the "brain" of the computer. More on it in 1.2.2.)
   [insert image about an input/output controller device]

3. **Input devices**: Tools that users interact with to give data to the computer. Each one handles a specific kind of data:

Device	What it does
Keyboard	Captures typed text
Mouse	Detects movements and clicks and turns them into cursor movement
Touchscreen	Combines input and output by detecting touches right on the display
Scanner	Turns paper documents into digital data
Microphone	Captures sound

4. **Input data**: The actual information being sent in. It can be:
    - **Text** (from a keyboard)
    - **Audio** (from a microphone)
    - **Images or video** (from a camera)
    - **Sensor signals** (such as temperature, light, or motion readings from a **sensor**, a device that measures something in the real world)

    Input data can also include commands, raw numbers, or touch gestures. The type and quality of the input data affect how the computer handles it.

WORKED EXAMPLE

You press the letter A on a keyboard.

Step	Element	What happens
1	Input device	The keyboard detects the key press
2	Input controller	The signal is prepared and passed along
3	Input interface	The USB port carries the signal to the computer
4	Input data	The computer receives the data "A"

ACTIVITY

For each situation, name the input device and the type of input data.

a. You speak into a headset during a video call.
b. You photograph a page of your notes with a phone camera.
c. A chip in a smart home reads the room temperature.

ANSWER

a. Input device: **microphone**. Input data: **audio**.
b. Input device: **camera**. Input data: **images**.
c. Input device: **sensor**. Input data: **sensor signals** (temperature).

1.6 Workflow of a Computer: Data Processing

Processing means working on the input data by following the program's instructions, usually using logic and arithmetic. The **central processing unit (CPU)** does most of this work. It is the **processing unit** of the computer.

Once the input interface delivers the data, the CPU:

- **Analyzes** the data and converts it into a form the computer can use.
- **Executes** (carries out) the needed operations. For example, if you press a key, the CPU recognizes it, converts it to binary, and triggers the matching action, like showing a letter on the screen.
- **Communicates** with memory, storage, and output devices to finish bigger tasks.

A computer is **programmable**, which means it can run different sets of instructions. This is what makes it useful for so many different jobs.

Data representation

Computers store data using the **binary system**. A **bit** (short for *binary digit*) is a single `0` or `1`. You can think of a bit like a light bulb: it is either **off (0)** or **on (1)**. In logic, `1` can also mean **True** and `0` can mean **False**.

[insert image about a light bulb off (0) and a light bulb on (1)]

One bit is too small to hold anything useful, so computers **string many bits together** into a line of 1s and 0s. Binary makes it easy for the hardware to store and work with information. From binary, computers can build bigger kinds of data, such as **integers** (whole numbers), **floating-point numbers** (numbers with a decimal point), and **characters** (letters and symbols).

Reading binary

Each bit has a **place value**, which is a **power of 2**. A power of 2 means 2 multiplied by itself a number of times (for example, $2^3 = 2 \times 2 \times 2 = 8$). Place values count **from right to left**, starting at 2^0:

Power	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
Value	128	64	32	16	8	4	2	1

To turn a binary number into a **decimal** number (the normal numbers we use every day), keep the values where the bit is **1** and add them up.

[insert image about binary place values 2^7 to 2^0 above a string of bits]

ASCII

To turn text into binary, computers use a code. **ASCII** (American Standard Code for Information Interchange) is a **character encoding** scheme, which means a system that gives every letter, digit, and symbol its own number. ASCII uses **7 bits** for each character, which gives 128 different combinations (from *0000000* to *1111111*, or 0 to 127 in decimal). The computer can "call" a character's ASCII number to display that character. For example, the letter **A** has the number **65**. Thanks to ASCII, the computer can turn binary into text that people can read.

Characters are usually grouped into a **character set**, which is the collection of characters a system can use. A character set includes:

- **Alphanumeric data**: letters and numbers.
- **Symbols**: such as *?*, *&*, and *∴*.
- **Control characters**: special characters that give commands instead of showing a symbol, such as Backspace, Horizontal Tab, and Escape.

WORKED EXAMPLE

Task 1: Convert *00010011* to decimal.

Bit	0	0	0	1	0	0	1	1
Place value	128	64	32	16	8	4	2	1
Kept?	No	No	No	**Yes**	No	No	**Yes**	**Yes**

16 + 2 + 1 = **19**

Task 2: How does the computer know *1000001* is the letter A?

1. Add the place values of the 1s: 64 + 1 = **65**
2. Look up 65 in the ASCII table: it is the character **A**.

ACTIVITY

a. Convert `00101101` to decimal.

b. What decimal number is `01000001`, and which character does it show?

c. What is the binary code for `B`? (Use the ASCII table in 1.1.3.)

d. Is Escape an alphanumeric character, a symbol, or a control character?

ANSWER

a. 32 + 8 + 4 + 1 = **45**
b. 64 + 1 = **65**, which is the letter **A**
c. `1000010`
d. A **control character**

1.7 Workflow of a Computer: Output

Output is the data or information that a computer sends to an outside device or to the user after it has processed the input. The output system has three parts:

1. **Output interface**: The part that sends processed data from inside the computer to the outside world (to a user or another device). It has **hardware** (the output devices) and **software** (called **drivers**, small programs that let the computer communicate with a device).
2. **Output devices**: Devices that take the processed data and show it in a form people can use:

Device	What it does
Monitor (display)	Turns digital data into pictures, text, or video
Printer	Makes a **hard copy** (a paper copy) of documents or images

3. **Digital-to-analog conversion**: Computers work with **digital signals** (made of 0s and 1s), but some devices, like speakers, need **analog signals** (smooth, continuously changing waves). A **sound card** in the computer converts digital audio files into analog waves that speakers can play. The part that does this job is a **digital-to-analog converter (DAC)**. Inside a DAC, the digital input is first held in a register, then turned into voltage levels, added together, boosted by an amplifier, and sent out as an analog signal.

[insert image about a digital-to-analog converter turning digital input into an analog output]

How the process works: After the CPU finishes processing, it sends the results to the correct output system. The data travels to the connected device, where it is displayed, printed, or played. The format of the output depends on the type of data and the device being used.

[insert image about the Input -> Process -> Output flow with Storage connected to Process]

WORKED EXAMPLE

Task: You play a song on your computer.

Step	What happens
1	The CPU processes the digital audio file
2	The CPU sends the data to the output system
3	The **sound card (DAC)** converts the digital audio to analog waves
4	The **speakers** play the sound

ACTIVITY

Name the best output device for each result, and say whether a digital-to-analog conversion is needed:
a. You want a paper copy of a form.
b. You want to watch a photo slideshow.
c. You want to hear a voice message.

ANSWER

a. Printer. No conversion is needed.
b. Monitor. No conversion is needed.
c. Speakers. Yes, a digital-to-analog conversion is needed to turn the digital audio into sound waves.

1.8 Workflow of a Computer: Storage

Storage is the part of a computer that **saves and keeps data**. There are two-time scales:

- **Primary storage** (short term): Fast memory the computer uses while it is working, such as **RAM** (Random Access Memory). It is usually **volatile**, which means its contents are lost when the power is turned off.
- **Secondary storage** (long term): Storage that keeps data even when the computer is powered off. This is called **non-volatile** storage. Examples are a **hard disk drive (HDD)**, which stores data on spinning magnetic disks, and a **solid-state drive (SSD)**, which stores data on electronic chips with no moving parts.

Storage holds both **programs** and **data**. In the von Neumann model, storage keeps the program's instructions and the data those instructions work on. In modern computers, everything is stored in **binary**, so the same patterns of 0s and 1s can stand for text, images, or videos.

Feature	Primary storage (RAM)	Secondary storage (HDD / SSD)
Time scale	Short term	Long term
Speed	Very fast	Slower
Keeps data when powered off?	No	Yes
Used for	Programs and data currently in use	Files you want to keep

[insert image about primary storage (RAM) and secondary storage (a drive) in a computer]

WORKED EXAMPLE

You edit a photo and save it.

Step	Where is the photo?
1	You open the photo: it is loaded into **primary storage (RAM)** so you can edit it quickly
2	You click Save: it is copied to **secondary storage** (for example, the SSD)
3	You switch off the computer: RAM is cleared, but the saved photo remains in secondary storage

ACTIVITY

Is each example primary or secondary storage?
a. The place where a game is held while it is running.
b. A drive that keeps your documents after the computer is turned off.
c. Storage that loses its contents when the power is off.

ANSWER

a. Primary storage (RAM)
b. Secondary storage
c. Primary storage (RAM)

1.9 Different Types of Computers

Today, there are many kinds of computers, each made for different needs and jobs. They all follow the same basic workflow (input, processing, output, and storage).

1. Personal Computers (PCs)

- **Desktop computers**: Traditional computers with a separate monitor, keyboard, and CPU tower. They stay in one place and usually offer high processing power.
- **Laptop computers**: Portable computers where the screen, keyboard, and CPU are built into one unit. They balance performance with the ability to carry them around.

[insert image about a desktop computer setup]

2. Mobile Devices

- **Smartphones**: Pocket-sized devices with powerful computing abilities. They make calls and send messages, browse the Internet, and run many kinds of apps.
- **Tablets**: Larger touchscreen devices that sit between a smartphone and a PC. They are great for casual browsing, watching media, and light work.

[insert image about a smartphone and a tablet]

3. Servers and Supercomputers

- **Servers**: Specialized computers that manage resources and provide services to other devices on a **network** (a group of connected computers that share data). They are used for **web hosting** (keeping websites online), **database management** (storing and organizing large amounts of data), and network communication.
- **Supercomputers**: Extremely fast systems built for very complex calculations and huge amounts of data. They are used for scientific simulations, climate modeling, and advanced data analysis.

[insert image about a server room and a supercomputer]

4. Embedded Systems

Embedded systems are small, specialized computers built into everyday objects, such as appliances, cars, factory machines, and medical equipment. Each one performs a specific job and is important for automation, control, and monitoring.

[insert image about an embedded system, such as a car engine management module]

Type	Main purpose	Examples
Desktop	Powerful, stationary use	Office computer
Laptop	Portable everyday computing	School or work laptop
Smartphone	Calls, messaging, apps in your pocket	Phone
Tablet	Browsing and media on a touchscreen	Tablet
Server	Provides services to other devices	Web server
Supercomputer	Huge, complex calculations	Climate model
Embedded system	One built-in job inside a device	Car engine controller, washing machine

WORKED EXAMPLE

Pick the best type of computer for each person.

Person's need	Best type	Why
A video editor needs strong power at a fixed desk	Desktop	High processing power for stationary use
A student takes notes in different classrooms	Laptop	Portable, with a built-in screen and keyboard
A company runs a website for thousands of visitors	Server	Provides services to many devices on a network
A scientist models the climate	Supercomputer	Handles very complex calculations at high speed

ACTIVITY

Name the type of computer in each situation:
a. The small computer that controls the temperature inside a smart oven.
b. A touchscreen device used for reading and watching videos on the couch.
c. A computer that stores and shares your school's files with every student.
d. A system used to simulate how a storm will move across the ocean.

ANSWER

a. Embedded system
b. Tablet
c. Server
d. Supercomputer

STAGE 1 SUMMARY

- A computer takes data (input), processes it using a program, and gives a result (output). It can also save data (storage).
- The Turing model shows that a computer's output depends on its input data and its program.
- The von Neumann model splits a computer into Memory, ALU, Control Unit, and Input/Output, and stores programs and data together.
- Computers have hardware (physical parts), software (programs), and data.
- Computers use binary (0s and 1s). Each bit has a place value, and ASCII turns characters into binary codes.
- The basic workflow is Input → Processing → Output, with Storage saving data along the way.
- Output devices such as monitors, printers, and speakers present results, and speakers need digital-to-analog conversion.
- Primary storage (RAM) is fast and short term. Secondary storage (HDD/SSD) keeps data long term.
- Computers come in many types: PCs, mobile devices, servers, supercomputers, and embedded systems.

Stage 2: Number Systems

LEARNING GOALS

By the end of this stage, you will be able to:

1. Identify core physical components of computers and explain basic functions. (Computers use binary because their hardware has two states, on and off. Number systems are how we read and write that data.)
2. Explain what a number system is and describe the binary, decimal, and hexadecimal systems.
3. Convert between binary and decimal.
4. Convert between hexadecimal and binary.
5. Convert between hexadecimal and decimal.

TOPICS COVERED

2.1 2.1 Number System Overview

2.2 2.2 Binary Number System

2.3 2.3 Decimal to Binary and Binary to Decimal

2.4 2.4 Hexadecimal to Binary and Binary to Hexadecimal

2.5 2.5 Hexadecimal to Decimal and Decimal to Hexadecimal

2.1 Number System Overview

A **number system** is a way of writing numbers using a set of symbols, called **digits**. Every number system has a **base**. The base tells you **how many different digits** the system uses before it "rolls over" to the next place. (In everyday counting, we roll over after 9: we write 10, which means "one group of ten and zero ones".)

The most common number systems in computing are:

System	Base	Digits used	Example
Binary	2	0, 1	*1101*
Octal	8	0 to 7	*157*
Decimal	10	0 to 9	*106*
Hexadecimal	16	0 to 9 and A to F	*6A*

All of these are **positional number systems**. That means the **position** of a digit decides its value. Each position has a **place value**, which is a **power of the base**. A **power** means the base multiplied by itself a number of times (for example, $2^3 = 2 \times 2 \times 2 = 8$, and any base to the power of 0 equals 1).

When two systems are mixed in the same text, a small number written after the value shows the base. For example, 157_8 is in octal and 157_{10} is in decimal.

The **octal** system (base 8) is less common, but it is a short way to write binary, because 8 is a power of 2.

[insert image about number systems: binary, octal, decimal, and hexadecimal side by side]

WORKED EXAMPLE

Find the value of the decimal number *352*, and the octal number 157_8.

1) Decimal *352* (base 10)

Digit	3	5	2
Place value	$10^2 = 100$	$10^1 = 10$	$10^0 = 1$
Digit × place value	300	50	2

300 + 50 + 2 = **352**

2) Octal 157_8 (base 8):

Digit	1	5	7
Place value	$8^2 = 64$	$8^1 = 8$	$8^0 = 1$
Digit × place value	64	40	7

64 + 40 + 7 = **111** in decimal, so $157_8 = 111_{10}$

ACTIVITY

a. What is the base of the binary system, and which digits does it use?
b. How many different digits does the hexadecimal system use?
c. Find the decimal value of the octal number 24_8.

ANSWER

a. Base 2. It uses the digits 0 and 1.
b. 16 digits (0 to 9 and A to F).
c. (2 × 8) + (4 × 1) = 16 + 4 = 20

2.2 Binary Number System

The **binary number system** (base 2) uses only two digits: *0* and *1*. A single binary digit is called a **bit**. Binary fits computers well, because the hardware inside only has two states: **off (0)** and **on (1)**, like a light bulb.

Each place in a binary number is a **power of 2**. Place values count **from right to left**, starting at 2^0:

Power	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
Place value	128	64	32	16	8	4	2	1
Power	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
Place value	128	64	32	16	8	4	2	1
Power	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0

Two useful terms:

- **Most Significant Bit (MSB)**: The leftmost bit. It has the biggest place value.
- **Least Significant Bit (LSB)**: The rightmost bit. It has the smallest place value.

In logic, a `1` can also mean **True** and a *0* can mean **False**.

[insert image about binary place values 2^7 to 2^0 above a string of bits, with MSB and LSB labeled]

WORKED EXAMPLE

Write out the place value of each bit in *00010011*, and say which bits are True.

Bit	0	0	0	1	0	0	1	1
Place value	128	64	32	16	8	4	2	1
True / False	False	False	False	**True**	False	False	**True**	**True**

- The **MSB** is the leftmost `0` (place value 128).
- The **LSB** is the rightmost `1` (place value 1).

> **ACTIVITY**
> Look at the binary number 10100110.
> a. What is the MSB, and what is its place value?
> b. What is the LSB, and what is its place value?
> c. Which place values are True (bit = 1)?

ANSWER

a. The MSB is 1, with place value 128.
b. The LSB is 0, with place value 1.
c. The bits that are 1 are at place values 128, 32, 4, and 2.

2.3 Decimal to Binary and Binary to Decimal

BINARY TO DECIMAL

To turn a binary number into a **decimal** number:

1. Write the place value above each bit.
2. Keep the place values where the bit is **1** (True). Ignore the ones where the bit is **0** (False).
3. **Add** the kept values.

WORKED EXAMPLE

Task 1: Convert *00010011* to decimal.

Bit	0	0	0	1	0	0	1
Place value	128	64	32	16	8	4	2
Kept?	No	No	No	**Yes**	No	No	**Yes**

16 + 2 + 1 = **19**

Task 2: Convert `1101010` to decimal.

$1 \times 2^6 + 1 \times 2^5 + 0 \times 2^4 + 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 0 \times 2^0$
= 64 + 32 + 0 + 8 + 0 + 2 + 0
= **106**

ACTIVITY

Convert to decimal:
a. 11001
b. 1011011

ANSWER

a. 16 + 8 + 0 + 0 + 1 = 25
b. 64 + 0 + 16 + 8 + 0 + 2 + 1 = 91

DECIMAL TO BINARY

To turn a decimal number into binary, use **repeated division by 2**:

1. Divide the number by 2. Write down the **remainder** (what is left over, either 0 or 1).
2. Divide the result (the **quotient**) by 2 again, and write the new remainder.

3. Keep going until the quotient is **0**.
4. Read the remainders **from the last to the first** (bottom to top). That is the binary number.

[insert image about repeated division by 2 with remainders read from bottom to top]

WORKED EXAMPLE

Convert *19* to binary.

Division	Quotient	Remainder
19 ÷ 2	9	**1**
9 ÷ 2	4	**1**
4 ÷ 2	2	**0**
2 ÷ 2	1	**0**
1 ÷ 2	0	**1**

Read the remainders from bottom to top: *10011*

Add zeros on the left to fill 8 bits (this does not change the value): *00010011*

Check: 16 + 2 + 1 = 19 ✓

> **ACTIVITY**
> Convert to binary:
> a. 45
> b. 100

ANSWER

a. 45 ÷ 2 = 22 r 1, 22 ÷ 2 = 11 r 0, 11 ÷ 2 = 5 r 1, 5 ÷ 2 = 2 r 1, 2 ÷ 2 = 1 r 0, 1 ÷ 2 = 0 r 1. Reading from the bottom: 101101 (check: 32 + 8 + 4 + 1 = 45).
b. 100 ÷ 2 = 50 r 0, 50 ÷ 2 = 25 r 0, 25 ÷ 2 = 12 r 1, 12 ÷ 2 = 6 r 0, 6 ÷ 2 = 3 r 0, 3 ÷ 2 = 1 r 1, 1 ÷ 2 = 0 r 1. Reading from the bottom: 1100100 (check: 64 + 32 + 4 = 100).

2.4 Hexadecimal to Binary and Binary to Hexadecimal

The **hexadecimal system**, or **hex** (base 16), uses sixteen digits: `0` to `9`, then letters for the larger values:

Hex digit	A	B	C	D	E	F
Decimal value	10	11	12	13	14	15

Hex is very popular in computing because:

- **It is compact**: A long binary number becomes much shorter. For example, a 16-bit binary number can be written with just 4 hex digits.
- **It is easy to read**: People can read and write hex more easily than long strings of 1s and 0s.
- **It is common in programming**: Memory addresses, color codes in web design (like `#FF0000` for red), and machine code instructions are often written in hex.

There are two ways to convert between binary and hex:

- **Indirect method**: First convert to another system (such as decimal), then convert to the final system.
- **Direct method (grouping)**: Because there are 16 hex digits, each hex digit can be written with exactly **4 bits**. So you can split the binary number into groups of 4 bits and swap each group for a hex digit.

Hex and binary table:

Hex	0	1	2	3	4	5	6	7
Binary	0000	0001	0010	0011	0100	0101	0110	0111
Hex	8	9	A	B	C	D	E	F
Binary	1000	1001	1010	1011	1100	1101	1110	1111

Binary to hex (grouping):

1. Start at the **right** end and split the bits into groups of 4.
2. If the leftmost group has fewer than 4 bits, add **zeros on the left** to complete it. This does not change the value, just like 007 is the same as 7.
3. Replace each group with its hex digit from the table.

Hex to binary: Replace each hex digit with its 4-bit group from the table.

[insert image about grouping a binary number into 4-bit groups and replacing each group with a hex digit]

WORKED EXAMPLE

Task 1 (indirect method): Convert `1101010` to hex.

1. Convert to decimal: 64 + 32 + 0 + 8 + 0 + 2 + 0 = **106**
2. Convert 106 to hex: 106 = (6 × 16) + 10, and 10 is written as A. So the answer is $\mathbf{6A_{16}}$.

Task 2 (grouping method): Convert `1010101101001` to hex.

1. Split into groups of 4 from the right: `1 | 0101 | 0110 | 1001`
2. Add three zeros on the left to fill the first group: `0001 | 0101 | 0110 | 1001`
3. Replace each group: 0001 = **1**, 0101 = **5**, 0110 = **6**, 1001 = **9**

Answer: $\mathbf{1569_{16}}$

Task 3 (hex to binary): Convert $3F2_{16}$ to binary.

3 = `0011`, F = `1111`, 2 = `0010`

Answer: `001111110010` (the leading zeros can be dropped: `1111110010`)

> **ACTIVITY**
> a. Use grouping to convert `10111100` to hex.
> b. Use grouping to convert `101111` to hex. (Remember to fill the first group with zeros.)
> c. Convert $A5_{16}$ to binary.

ANSWER
a. `1011` | `1100` → B and C → **BC**
b. Add zeros: `0010` | `1111` → 2 and F → **2F**
c. A = `1010`, 5 = `0101` → **10100101**

2.5 Hexadecimal to Decimal and Decimal to Hexadecimal

HEXADECIMAL TO DECIMAL

Hex numbers use **powers of 16** as place values. Positions are counted from right to left, starting at 16^0:

Power	16^3	16^2	16^1	16^0
Place value	4096	256	16	1

Hex can also have a **fractional part** (the part after the hex point, like the decimal point). Those places count from left to right as $16^{-1} = 1/16$, $16^{-2} = 1/256$, $16^{-3} = 1/4096$, and so on.

To convert hex to decimal:

1. Change each hex letter to its decimal value (A = 10, B = 11, and so on).
2. Multiply each digit by its place value.
3. Add the results.

[insert image about hexadecimal place values 16^3, 16^2, 16^1, 16^0 and the fractional places]

WORKED EXAMPLE

Convert $3F2_{16}$ to decimal. (F = 15)

Digit	3	F	2
Place value	$16^2 = 256$	$16^1 = 16$	$16^0 = 1$
Digit × place value	3 × 256 = 768	15 × 16 = 240	2 × 1 = 2

768 + 240 + 2 = **1010**

> **ACTIVITY**
> Convert to decimal:
>
> a. $2F_{16}$
> b. $A5_{16}$
> c. $1C_{16}$

ANSWER

Convert to decimal:

a. $2F_{16}$
b. $A5_{16}$
c. $1C_{16}$

DECIMAL TO HEXADECIMAL

To convert a decimal number to hex, use **repeated division by 16**:

1. Divide the number by 16 and write down the **remainder**. If the remainder is 10 to 15, write it as A to F.
2. Divide the quotient by 16 again, and write the new remainder.
3. Keep going until the quotient is **0**.
4. Read the remainders **from last to first** (bottom to top).

WORKED EXAMPLE

Task 1: Convert 106 to hex.

Division	Quotient	Remainder
106 ÷ 16	6	10 → A
6 ÷ 16	0	6

Read from bottom to top: $\mathbf{6A_{16}}$ (this matches the answer from section 2.4).

Task 2: Convert `1010` to hex.

Division	Quotient	Remainder
1010 ÷ 16	63	**2**
63 ÷ 16	3	15 → **F**
3 ÷ 16	0	**3**

Read from bottom to top: $\mathbf{3F2_{16}}$

ACTIVITY

Convert to hex:

a. `255`
b. `300`

ANSWER

a. 255 ÷ 16 = 15 r 15 (F), 15 ÷ 16 = 0 r 15 (F). Reading from the bottom: **FF**
b. 300 ÷ 16 = 18 r 12 (C), 18 ÷ 16 = 1 r 2, 1 ÷ 16 = 0 r 1. Reading from the bottom: **12C** (check: 256 + 32 + 12 = 300)

STAGE 2 SUMMARY

- A **number system** writes numbers using digits. Its **base** tells how many digits it uses before rolling over.
- **Binary** (base 2) uses 0 and 1, **octal** (base 8) uses 0 to 7, **decimal** (base 10) uses 0 to 9, and **hexadecimal** (base 16) uses 0 to 9 and A to F.
- In a **positional** number system, each place has a value that is a **power of the base**.
- **Binary to decimal**: Add the place values of the 1s.
- **Decimal to binary**: Divide by 2 again and again, then read the remainders from bottom to top.
- **Binary to hex**: Group the bits in fours (adding zeros on the left if needed) and replace each group with a hex digit. **Hex to binary**: Replace each hex digit with its 4 bits.
- **Hex to decimal**: Multiply each digit by its power of 16 and add. **Decimal to hex**: Divide by 16 again and again, then read the remainders from bottom to top.

Stage 3: Data Storage

LEARNING GOALS

By the end of this stage you should be able to:

1. Identify the core physical components of a computer that store data and explain their basic functions.
2. Explain how data is stored and retrieved inside a computer.
3. Describe how numbers, text, images, audio and video are represented in binary.
4. Compare primary and secondary storage.
5. Compare common storage devices (HDD, SSD, optical discs, USB drives) and choose the right one for a job.

TOPICS COVERED

3.1 3.1

3.2 3.2 How Does Data Storage Work?

3.3 Representing Data in Binary: 3.3 Representing Data in Binary: Numbers

3.4 Representing Data in Binary: 3.4 Representing Data in Binary: Text

3.5 Representing Data in Binary: 3.5 Representing Data in Binary: Images

3.6 Representing Data in Binary: 3.6 Representing Data in Binary: Audio

3.7 Representing Data in Binary: 3.7 Representing Data in Binary: Video

3.8 3.8. Types of Data Storage: Primary

3.9 **Error! Reference source not found.**

3.10 3.9 Types of Data Storage: Secondary Storage

3.11 3.10 Data Storage Devices (HDD, SSD, USB Drives)

3.1 What is Data Storage?

Data is any information a computer can work with, such as a number, a word, a photo or a song.

Storage is the part of a computer that holds data so the **CPU** (Central Processing Unit, the "brain" that carries out instructions) and other components can reach it when they need it.

Storage is not one single part. A computer uses **several kinds** of storage together:

- Some storage is **very fast** but small and temporary.
- Some storage is **slower** but large and permanent.

Two words you will see often:

Term	Meaning
Volatile	Loses its data when the power is turned off.
Non-volatile	Keeps its data even when the power is off.

WORKED EXAMPLE

You are writing an essay.

- While you type, the text sits in a fast, temporary storage area. If the power suddenly cuts out, the unsaved text is gone. This storage is **volatile**.
- When you press **Save**, the essay is copied to a permanent storage area. You can switch the computer off and open the essay tomorrow. This storage is **non-volatile**.

ACTIVITY

Write **V** (volatile) or **N** (non-volatile) for each situation:

1. A web page you have open but have not saved.
2. A photo saved on a USB flash drive.
3. A game installed on your computer's drive.
4. The numbers a calculator program is using right now.

ANSWER

1. **V**, open and unsaved data lives in temporary storage.
2. **N**, a USB drive keeps data without power.
3. **N**, installed programs are kept permanently.
4. **V**, data in use right now is held in temporary storage.

3.2 How Does Data Storage Work?

A computer moves data between storage areas all the time. A simplified journey looks like this:

1. **Secondary storage** (permanent) holds your files and programs.
2. When you open something, the data is copied into **RAM** (fast, temporary memory).
3. The CPU reads from RAM, because it is much quicker than reading from permanent storage.
4. When you save, the data is copied back to secondary storage.

RAM acts as a **high-speed buffer** (a waiting area) between the CPU and slower storage devices such as hard drives. It lets the computer quickly find and process the data needed to run applications and carry out commands.

Where does the data travel? Through the **motherboard**, the main circuit board that connects all the parts. Two things on the motherboard affect how smoothly memory and CPU "talk" to each other:

- The **chipset**, a group of chips that manages communication between components.

- The **bus speed**, how fast data can travel along the connecting pathways (the **bus**).

The motherboard also has **memory slots** where RAM modules are plugged in.

WORKED EXAMPLE

You double-click a video game.

Step	What happens
1	The game's files are found on the SSD or hard drive (secondary storage).
2	The parts of the game that are needed are copied into RAM.
3	The CPU reads the game's instructions from RAM and runs them.
4	When you save your progress, a save file is written back to the drive.

ACTIVITY

Put these steps in the correct order:

- The CPU processes the data in RAM.
- The file is saved back to the drive.
- You open a file stored on the drive.
- D. The file's data is copied into RAM.

ANSWER

C → D → A → B
You open the file (C), it is copied to RAM (D), the CPU works on it (A), then it is saved back (B).

3.3 Representing Data in Binary: Numbers

Computers are built from electronic switches that are either **on** or **off**. This is why they use **binary**, a number system with only two digits: **0** and **1**.

- A **bit** (binary digit) is a single 0 or 1.
- A **byte** is a group of **8 bits**.

Everything stored in a computer (numbers, text, images, audio, video) is turned into long strings of bits.

Storage units (we use 1000 for simplicity; some systems use 1024):

Unit	Size
1 byte (B)	8 bits
1 kilobyte (KB)	1,000 bytes
1 megabyte (MB)	1,000 KB
1 gigabyte (GB)	1,000 MB
1 terabyte (TB)	1,000 GB

Binary is a **place-value** system. In everyday (decimal) numbers the places are ones, tens, hundreds. In binary the places are **powers of 2**:

Place value	128	64	32	16	8	4	2	1

To write a number in binary, put a **1** under each place value you need and a **0** under the rest.

WORKED EXAMPLE

Convert **13** to binary.

- 13 = 8 + 4 + 1
- Place values: 8 → 1, 4 → 1, 2 → 0, 1 → 1

8	4	2	1
1	1	0	1

13 in binary = 1101 (or `00001101` in a full byte).

> **ACTIVITY**
> Convert 10 to binary.

ANSWER

10 = 8 + 2 → 1010.

3.4 Representing Data in Binary: Text

Computers cannot store letters directly, so every character gets a number. A **character code** is a table that links characters to numbers. A common one is **ASCII** (American Standard Code for Information Interchange). The number is then stored in binary.

Character	ASCII number	Binary

A	65	01000001
B	66	01000010
a	97	01100001

Each character uses 1 byte in basic ASCII.

WORKED EXAMPLE

Store the word **"Hi"**.

- H = 72 → `01001000`
- i = 105 → `01101001`

"Hi" = `01001000 01101001`, which is **2 bytes**.

> **ACTIVITY**
>
> 1. The ASCII number for **C** is 67. Write it in binary.
> 2. How many bytes does the word **"CAT"** need in ASCII?

ANSWER

1. 67 = 64 + 2 + 1 → **01000011**
2. 3 characters × 1 byte = **3 bytes**.

3.5 Representing Data in Binary: Images

A digital image is a grid of tiny squares called **pixels**. Each pixel stores a colour as a binary number.

- **Resolution:** the number of pixels (width × height).
- **Colour depth:** the number of bits used for each pixel. More bits means more possible colours.

Colour depth	Colours possible
1 bit	2 (black and white)
8 bits	256
24 bits	about 16.7 million

File size = number of pixels × colour depth (in bits). Divide by 8 to get bytes.

WORKED EXAMPLE

A small image is **100 × 100 pixels** with **24-bit** colour.

- Pixels: 100 × 100 = 10,000
- Bytes per pixel: 24 ÷ 8 = 3

- Size: 10,000 × 3 = **30,000 bytes = 30 KB**

> **ACTIVITY**
> A black-and-white (1-bit) image is **8 × 8 pixels**. How many bytes does it need?

> **ANSWER**
> 8 × 8 = 64 pixels × 1 bit = 64 bits ÷ 8 = **8 bytes**.

3.6 Representing Data in Binary: Audio

Sound is a smooth wave. To store it, the computer measures the wave many times per second. Each measurement is called a **sample**.

- **Sample rate:** how many samples are taken each second (measured in **Hz**).
- **Bit depth:** how many bits are used for each sample.

More samples and more bits give better quality, but a bigger file.

File size = sample rate × bit depth × seconds (in bits; divide by 8 for bytes).

WORKED EXAMPLE
One second of mono audio, **44,100 Hz**, **16 bits**:

- 44,100 × 16 = 705,600 bits
- 705,600 ÷ 8 = **88,200 bytes** (about 88 KB)

> **ACTIVITY**
> A voice recording uses 8,000 Hz and 8 bits per sample. How big is 10 seconds?

> **ANSWER**
> 8,000 × 8 × 10 = 640,000 bits ÷ 8 = 80,000 bytes = 80 KB.

3.7 Representing Data in Binary: Video

A video is a fast series of still images called **frames**, played together with audio.

- **Frame rate:** frames shown per second (**fps**).

A video's size grows quickly because it stores many images every second. Most videos are **compressed**, meaning the file is made smaller by removing or simplifying data, so they take far less space than the raw calculation suggests.

Uncompressed size ≈ size of one frame × frames per second × seconds (plus audio).

WORKED EXAMPLE
Each frame is 1 MB and the video runs at 10 fps.

- 1 second = 10 × 1 MB = 10 MB
- 3 seconds = **30 MB**

ACTIVITY

Each frame is 2 MB and the video runs at 5 fps. How big is **4 seconds** of uncompressed video?

ANSWER

2 MB × 5 fps × 4 s = 40 MB.

3.8. Types of Data Storage: Primary Storage

Primary storage is memory the CPU can reach directly and very quickly. It includes RAM, ROM and cache.

RAM (Random Access Memory)

- Holds data and programs **while the computer is running**.
- Very fast.
- **Volatile:** everything is lost when the power is off.
- Comes as modules plugged into the motherboard, such as **DDR4** or **DDR5** (types of RAM with different speeds and capacities).

ROM (Read-Only Memory)

- Holds data that **cannot be easily changed**.
- **Non-volatile:** keeps its data without power.
- In modern computers it usually holds **firmware**, software built into hardware that tells the device how to start up and work.

Cache memory

- A **smaller, faster** type of volatile memory placed **close to the CPU**.
- Stores **frequently used** information so the CPU does not have to wait for RAM.

WORKED EXAMPLE

When you switch on a laptop:

1. **ROM** provides the firmware that starts the computer.
2. The system loads into **RAM**.
3. Instructions the CPU uses again and again are kept in **cache** for speed.

ACTIVITY

Match each description to **RAM**, **ROM** or **Cache**:

1. Keeps the start-up firmware, even when the power is off.
2. Holds the programs you have open right now.

Small, very fast memory next to the CPU for frequently used data.

ANSWER

1. ROM
2. RAM
3. Cache

3.9 Types of Data Storage: Secondary Storage

Secondary storage is for **permanent** storage. It includes hard drives (HDD), solid-state drives (SSD), optical drives and USB flash drives.

- **Non-volatile:** data stays when the computer is off.
- **Larger capacity** than primary storage.
- **Slower** than primary storage.
- Used for documents, system files, applications and databases.

PRIMARY VS SECONDARY AT A GLANCE

Feature	Primary storage (RAM)	Secondary storage
Speed	Very fast	Slower
Capacity	Smaller	Much larger
Volatile?	Yes (RAM)	No
Purpose	Data in use right now	Long-term saving
Examples	RAM, cache, ROM	HDD, SSD, USB drive, DVD

WORKED EXAMPLE

Why not just save everything in RAM, since it is so fast?

Because RAM needs constant power, everything would vanish when you switch off. So data that is not being used right now is **moved to secondary storage**, which is slower but keeps it safe.

> **ACTIVITY**
> A student has a **16 GB RAM** laptop and a **512 GB SSD**. Which one holds her photos after she shuts down the laptop, and why?

ANSWER

The **SSD**. It is secondary storage, so it is non-volatile and keeps the photos without power. RAM is volatile and would lose them.

3.10 Data Storage Devices (HDD, SSD, USB Drives)

Device	How it works	Main points
Hard Disk Drive (HDD)	Stores data magnetically on **spinning platters** (disks).	Large capacity, traditional, has moving parts.
Solid-State Drive (SSD)	Uses **flash memory** (chips that keep data without power).	Much faster than HDD, **no moving parts**.
Optical storage	A laser reads data from discs such as **CDs, DVDs and Blu-rays**.	Used for media or backup files.
USB flash drive	A small **solid-state** device that connects through a **USB port**.	Portable; capacities from a few GB up to several TB.

USB DRIVES IN MORE DETAIL

USB drives are commonly used to **transfer files** between devices and for **backup**.

Advantages

- **Portable:** small and light, easy to carry.
- **Plug-and-play:** no installation needed. Plug it into a USB port and it is ready to use.

Disadvantages

- **Limited lifespan:** like all flash storage, a USB drive can only be written to a limited number of times (called **write cycles**). Most people will never reach this limit unless they use the drive heavily.
- **Easy to lose or damage:** its small size means it can be misplaced or damaged if handled carelessly.

Data access and retrieval

- **Primary storage (RAM)** is much faster than secondary storage, so the CPU can get data quickly. It needs a constant power supply, which is why data not in use is moved to secondary storage.
- **Secondary storage** offers more capacity and long-term storage, but at slower speeds.

WORKED EXAMPLE

A photographer needs to carry 20 GB of photos to a friend's house.

- Needs: portable, easy to use on any computer, enough capacity.
- Best choice: a **USB flash drive** (e.g. 64 GB). It is small, plug-and-play, and 64 GB is plenty.
- Why not an HDD? Less convenient to carry and has moving parts that can be damaged by knocks.

Extra check: if each photo is 4 MB, how many fit on a 64 GB drive? 64 GB = 64,000 MB, and 64,000 ÷ 4 = **16,000 photos**.

ACTIVITY

Choose the best device for each situation and give one reason:

1. A laptop that must start up and open programs as fast as possible.
2. A cheap way to keep 4 TB of rarely used old videos at home.
3. Quickly moving a school presentation from one computer to another.
4. Distributing a film to customers on a disc.

ANSWER

1. **SSD**, it is much faster and has no moving parts.
2. **HDD**, it offers large capacity at a lower cost per GB.
3. **USB flash drive**, it is portable and plug-and-play.
4. **Optical disc (DVD/Blu-ray)**, it is designed for storing and distributing media.

Chapter 1.1 Summary

KEY POINTS

- **Storage** holds data so the CPU and other components can access it.
- Computers store everything as **binary** (0s and 1s). 8 bits = 1 byte.
- **Numbers** use place values (powers of 2), **text** uses character codes like ASCII, **images** use pixels, **audio** uses samples, and **video** uses frames.
- **Primary storage** (RAM, ROM, cache) is fast and close to the CPU. RAM and cache are volatile.
- **Secondary storage** (HDD, SSD, optical, USB) is slower, larger and non-volatile.
- **HDD** = spinning magnetic platters. **SSD** = flash, faster, no moving parts. **USB drive** = portable flash storage.

CAN YOU DO THESE?

- □ Explain the difference between volatile and non-volatile storage.
- □ Describe the path of data from drive to RAM to CPU and back.
- □ Convert a small number to binary.
- □ Calculate the size of a simple image or sound.
- □ Compare RAM, ROM and cache.
- □ Compare HDD, SSD, optical and USB storage and pick one for a task.

GLOSSARY

Term	Definition
Algorithm	A step-by-step set of rules for solving a problem, planned before a program is written. Related: program, software.
ALU (Arithmetic Logic Unit)	The part of the CPU that performs arithmetic and log-ic operations, such as addition and comparison. Related: CPU, control unit.
Analog signal	A continuous signal, such as a sound wave, that can vary smoothly over time. Related: digital signal, DAC.
ASCII	A character encoding system that gives each character a numeric code. In basic ASCII, uppercase “A” is 65 and lowercase “a” is 97; each character uses 1 byte. Related: character encoding, binary.
Base	The number of distinct digits used in a positional number system. Binary is base 2, octal is base 8, decimal is base 10, and hexadecimal is base 16. Related: positional number system, binary, hexadecimal.

GLOSSARY	
Term	**Definition**
Binary	A base-2 number system that uses only the digits 0 and 1. Computers use binary because hardware switches have two states, often described as off and on. Related: bit, byte, base.
Bit	A single binary digit, either 0 or 1. It is the smallest unit of digital da-ta. Related: byte, binary.
Bit depth	The number of bits used to store each sample of digital audio, or each colour value in an image. More bits usually mean better quality but larger files. Related: sample rate, pixel, file size.
Byte	A group of 8 bits. File sizes and storage capacities are commonly measured in bytes. Related: bit, file size.
Cache memory	A small, very fast memory close to the CPU that keeps frequently used data. Cache is smaller and faster than RAM. Related: RAM, CPU, primary storage.
Character encoding	A system that maps characters, such as letters and digits, to numeric codes so they can be stored in binary. ASCII is one example. Related: ASCII, binary.
Computer	An electronic device that takes in input data, processes it by following a program, and produces output, with memory and storage used during the process. Related: input, processing, output, storage, program.
Control unit	The part of the CPU that fetches and decodes instructions and tells the other parts of the computer what to do and when. Related: CPU, ALU, fetch-decode-execute cycle.
CPU (Central Processing Unit)	The main processor of a computer, made of the con-trol unit and the ALU. It executes program instructions. Related: control unit, ALU, fetch-decode-execute cycle.
DAC (Digital-to-Analog Converter)	A component, often on the sound card, that con-verts digital audio into analog waves that speakers can play. Related: sound card, analog signal, output device.

GLOSSARY	
Term	**Definition**
Data	Values, facts, text, images, sound or other information that a computer stores and processes. Related: input, processing, output, storage.
Data storage	Keeping digital data on a device or medium so that it can be kept and used again later. Related: primary storage, secondary storage, RAM, SSD.
Decimal	The base-10 number system, using digits 0 to 9. People commonly use decimal in everyday life. Related: binary, hexadecimal, octal.
Desktop computer	A stationary computer designed for use at a fixed desk, usually with a separate monitor and keyboard. Related: laptop, tablet, server.
Device driver	Software that lets the computer communicate with and control a hardware device, such as a printer or monitor. Related: output device, input device, software.
Digital signal	Data represented as discrete binary values, usually 0s and 1s. Computers process digital signals. Related: analog signal, binary, DAC.
Embedded system	A small, specialized computer built into an everyday object or machine, such as a washing machine or air-conditioning controller, to do one spe-cific job. Related: computer types, hardware.
Fetch-decode-execute cycle	The basic cycle in which the CPU fetches an instruc-tion from memory, decodes what it means, and executes it. Related: CPU, control unit, ALU, program counter.
File size	The amount of storage a file needs, usually measured in bytes. For im-ages, size can be estimated from pixels × bytes per pixel; for audio, from sample rate × bit depth × seconds ÷ 8. Related: byte, pixel, sample rate, bit depth.
Firmware	Low-level instructions stored in ROM or similar non-volatile memory that help start up and control hardware. Related: ROM, BIOS/UEFI, non-volatile.

GLOSSARY	
Term	**Definition**
Frame	A single image in a video. A video is a series of frames shown one after another, usually with sound, all stored as binary data. Related: video, pixel, binary.
Hard copy	A printed paper copy of a document, produced by a printer. Related: printer, output device.
Hard disk drive (HDD)	A secondary storage device that stores data on spinning magnetic platters. It is non-volatile but has moving parts. Related: SSD, secondary storage, non-volatile.
Hardware	The physical parts of a computer that you can touch, such as the key-board, CPU, motherboard and monitor. Related: software, computer.
Hexadecimal	A base-16 number system using digits 0 to 9 and letters A to F. Each hexadecimal digit represents exactly 4 bits, so it is a compact way to write binary data. Related: binary, base, MSB, LSB.
Input	Data or signals that enter a computer from the outside world. Related: input device, processing, output.
Input device	A device that sends data into a computer, such as a keyboard, mouse, scanner, camera or microphone. Related: input, output device.
Laptop	A portable computer with the screen, keyboard and processing parts built into one unit, suitable for use in different places. Related: desktop computer, tablet.
Least Significant Bit (LSB)	The rightmost bit in a binary number. It has the smallest place value. Related: Most Significant Bit, binary.
Memory	The part of a computer that stores program instructions and data. In the von Neumann model, the program and data are stored together in the same memory. Related: RAM, ROM, von Neumann model, stored-program concept.

GLOSSARY	
Term	**Definition**
Most Significant Bit (MSB)	The leftmost bit in a binary number. It has the larg-est place value. Related: Least Significant Bit, binary.

Chapter 2: Hardware and the Von Neumann Model

Stage 1: Von Neumann Model

LEARNING GOALS

By the end of this stage, you will be able to:

1. Identify core physical components of computers and explain basic functions.
2. Describe the von Neumann model and why it matters.
3. Explain the fetch-decode-execute process.
4. Describe the CPU and its two parts, the control unit and the ALU.
5. Explain the roles of memory, input, and output.

TOPICS COVERED

1.1 1.1 Definition

1.2 1.2 Process: Fetch-Decode-Execute

1.3 1.3 Components (1): The CPU (Control Unit and ALU)

1.4 1.4 Components (2): Memory, Input, and Output

1.1 Definition

Early computers had a big problem. Every time they needed to do a new job, people had to **rewire** them by hand. This took a long time.

Around 1944–1945, a mathematician named **John von Neumann** came up with a better idea. He noticed that **programs and data are really the same kind of thing: information**. (A **program** is a list of instructions that tells the computer what to do, and **data** is the information the program works on.) So, he suggested that a program should be **stored in the computer's memory**, right next to the data it uses.

This idea created the **von Neumann model**, which is the base of **most modern computers**.

The model divides a computer into **four subsystems**. A **subsystem** is a smaller part that does one main job within the whole computer.

Subsystem	Its job
Memory	Stores both programs and data

Subsystem	Its job
Arithmetic Logic Unit (ALU)	Performs calculations and logic operations
Control Unit	Manages all the operations
Input/Output	Handles the exchange of data with the outside world

The big benefit: because the program and data sit in the **same memory**, a computer can **switch easily between tasks**, without anyone needing to rewire the machine.

[insert image about the von Neumann model: Input data enters Input/Output, which connects to the ALU, Memory, and Control Unit, and Output data leaves]

WORKED EXAMPLE

Explain how a computer changes from playing music to editing a document.

Old way (before von Neumann)	Von Neumann way
Engineers rewire the machine for the new job	The new program is loaded into memory
Slow and difficult	Fast and easy

In the von Neumann model, the computer simply loads the document-editing program into memory, and the control unit starts following its instructions.

> **ACTIVITY**
> a. What big idea did von Neumann add to computer design? b. Name the four subsystems of the von Neumann model. c. Why does storing programs in memory make switching tasks easier?

ANSWER

a. Programs should be **stored in the computer's memory**, together with the data. b. **Memory, Arithmetic Logic Unit (ALU), Control Unit, and Input/Output.** c. To change tasks, the computer just loads a different program into memory. No one has to **rewire** the machine.

1.2 Process: Fetch-Decode-Execute

In the von Neumann model, the computer follows its program **one instruction at a time, in order**. This keeps programs running in an organized and predictable way. (An **instruction** is a single step the computer can carry out, such as "add these two numbers".)

For each instruction, the computer repeats a three-step cycle called the **fetch-decode-execute cycle**:

Step	What happens	Who does it
Fetch	The next instruction is **fetched** (brought) from memory	Control unit
Decode	The instruction is **decoded**, which means its meaning is worked out and the right parts are told what to do	Control unit
Execute	The instruction is **executed** (carried out)	ALU, or another part such as input/output

After the execute step, the computer moves on to the **next instruction** and the cycle starts again. The computer remembers where it is in the program using a small storage spot called the **program counter**. The program counter holds the **address** (the location number in memory) of the next instruction.

[insert image about the fetch-decode-execute cycle as a loop between memory, control unit, and ALU]

WORKED EXAMPLE

A program in memory has two instructions:

Add 4 and 6, and store the result.

Output the result.

Cycle	Step	What happens
1	Fetch	The control unit gets "Add 4 and 6" from memory
1	Decode	The control unit works out that this means "add" and tells the ALU
1	Execute	The ALU calculates 4 + 6 = 10, and the result is stored in memory
2	Fetch	The program counter moves on, and the control unit gets "Output the result"
2	Decode	The control unit works out that this means "send to the screen"
2	Execute	The output part displays **10** on the screen

> **ACTIVITY**
>
> a. Put these steps in the correct order: **Execute**, **Fetch**, **Decode**. b. Which step brings an instruction from memory to the control unit? c. What does the program counter keep track of?

ANSWER

a. **Fetch → Decode → Execute** b. **Fetch** c. The **address of the next instruction** to run.

1.3 Components (1): The CPU (Control Unit and ALU)

The **Central Processing Unit (CPU)** is the "brain" of the computer. It is the part that **processes** data and carries out instructions. The CPU is made of **two main parts**:

- **Control Unit**
- **Arithmetic Logic Unit (ALU)**

CONTROL UNIT

The **control unit manages all the operations** in the computer. You can think of it as a **traffic controller** or a **manager**. It does not do the calculations itself. Instead, it:

- **Fetches** instructions from memory.
- **Decodes** them to understand what they mean.
- **Tells the other parts** (ALU, memory, input/output) what to do and when.

ARITHMETIC LOGIC UNIT (ALU)

The **ALU** is the part that does the **actual calculations and comparisons**. It performs two kinds of operations:

Kind	Meaning	Examples
Arithmetic operations	Math on numbers	Add, subtract, multiply, divide
Logic operations	Comparing values and making true/false decisions	Is 15 greater than 12? Are these two numbers equal?

So: **CPU = Control Unit + ALU**.

[insert image about the CPU containing the control unit and the ALU]

WORKED EXAMPLE

The program says: "If the player's score is more than 100, show 'You win!'" The score is 120.

Step	Part	What happens
1	Control unit	Fetches and decodes the instruction "compare score with 100"
2	Control unit	Sends the numbers 120 and 100 to the ALU
3	ALU	Performs a **logic operation**: is 120 > 100? The answer is **True**

Step	Part	What happens
4	Control unit	Sees the answer is True and tells the output part to show "You win!"

> **ACTIVITY**
> a. Put these steps in the correct order: **Execute**, **Fetch**, **Decode**. b. Which step brings an instruction from memory to the control unit? c. What does the program counter keep track of?

ANSWER
a. **Fetch → Decode → Execute** b. **Fetch** c. The **address of the next instruction** to run.

1.4 Components (2): Memory, Input, and Output

The other two subsystems of the von Neumann model are **memory** and **input/output**. Together with the CPU, they make up the computer's **hardware**.

Hardware means the **physical components** of a computer, the parts you can touch. The memory, the ALU, and the input/output parts are all hardware.

MEMORY

Memory is where the computer stores **both programs and data**, in the **same place**. Each location in memory has its own **address**, like a house number, so the control unit can find exactly the instruction or data it needs.

[insert image about memory holding Program and Data together, with address numbers]

INPUT

Input is the data or commands that the computer receives from users or devices. Examples: typing on a keyboard, moving a mouse, or speaking into a microphone.

OUTPUT

Output is the **result that the computer gives back** to the user. Examples: showing text on a screen (a display) or printing a page on a printer.

Together, **input and output let the user and the computer interact** with each other. In the von Neumann model, both are handled by the **Input/Output subsystem**.

Component	Main job	Example
Memory	Stores programs and data	The chips that hold your open apps
Input	Brings data into the computer	Keyboard, mouse, microphone
Output	Sends results out to the user	Screen, printer, speakers

[insert image about an input example: a keyboard sending data to the computer screen]

[insert image about an output example: the computer sending a document to a printer]

WORKED EXAMPLE

You type `5 + 3` into a calculator program and see the answer on the screen. Show how all four subsystems work together.

Step	Subsystem	What happens
1	**Input**	You type `5 + 3` on the keyboard, and the computer receives it
2	**Memory**	The numbers 5 and 3 and the "add" instruction are stored in memory
3	**Control unit**	Fetches and decodes the "add" instruction
4	**ALU**	Calculates 5 + 3 = 8
5	**Memory**	Stores the result, 8
6	**Output**	Displays **8** on the screen

ACTIVITY

a. What is hardware?
b. Which subsystem stores both programs and data?
c. Name one input device and one output device.
d. Put these steps in order for printing a typed letter: Printer produces the page, You type the letter, The CPU processes the text.

ANSWER

a. The **physical components** of a computer, such as memory, the ALU, and the input/output parts.
b. **Memory**
c. For example, input device: **keyboard** (or mouse or microphone). Output device: **printer** (or screen or speakers). d. **You type the letter → The CPU processes the text → The printer produces the page**

STAGE 1 SUMMARY

- The **von Neumann model** says that programs and data are both information, so they can be **stored together in memory**. This lets computers switch tasks easily without being rewired.
- The model has four subsystems: **Memory**, **ALU**, **Control Unit**, and **Input/Output**.
- The computer follows instructions one at a time using the **fetch-decode-execute cycle**: fetch the instruction from memory, decode what it means, and execute it.
- The **CPU** is made of the **control unit** (the manager that directs everything) and the **ALU** (which does arithmetic and logic operations).

- **Memory** stores programs and data. **Input** brings data in, and **output** gives results back to the user.
- **Hardware** is the physical part of a computer.

Stage 2: Computer Hardware

LEARNING GOALS

By the end of this stage, you will be able to:

1. Identify core physical components of computers and explain basic functions.
2. Define hardware and tell the difference between internal and external hardware.
3. Describe common input devices: mouse, keyboard, microphone, camera, and touchpad.
4. Describe common output devices: monitor, printer, and speaker.
5. Explain the role of the main internal parts: motherboard, CPU, RAM, ROM, graphics card, and power supply.
6. Describe ports and external storage such as USB drives and memory cards.

Topics covered:

- Hardware Definition
- Input Devices: Mouse, Keyboard, Microphone, Camera, Touchpad
- Output Devices: Monitor, Printer, Speaker
- Internal Hardware: Motherboard, CPU, RAM, RAM vs ROM, Graphics Card, Power Supply
- Ports and External Storage: USB, Memory Card

2.1 Hardware Definition

Hardware means the **physical parts of a computer**, the parts you can see and touch. The programs that run on the hardware are called **software**.

Hardware can be split into two groups:

Group	Meaning	Examples
Internal hardware	Parts **inside** the computer case	Motherboard, CPU, RAM, graphics card, power supply
External hardware	Parts **outside** the case that connect to the computer. These are also called **peripherals**	Keyboard, mouse, monitor, printer, speakers

Hardware can also be sorted by its job:

- **Input devices** send data **into** the computer.
- **Output devices** show or deliver results **out of** the computer.
- **Internal hardware** processes, stores, and moves the data.
- **Ports and external storage** connect devices and keep data outside the main case.

[insert image about the main hardware of a computer: monitor, keyboard, mouse, and the case with internal parts]

WORKED EXAMPLE

Task: Sort each item into **internal** or **external** hardware.

Item	Group
Keyboard	External
CPU	Internal
Printer	External
RAM	Internal

ACTIVITY

a. What is hardware? b. Is a mouse internal or external hardware? c. Is a word processor program hardware or software?

Answer

a. The **physical parts** of a computer that you can touch. b. **External** (it is a peripheral that connects to the computer). c. **Software** (it is a program, not a physical part).

2.1 Input Devices

Input devices are the devices used to **give data and commands to the computer**.

Device	What it does
Keyboard	One of the most common input devices. Each key matches a **character** (a letter, number, or symbol), a **function**, or a **command**. When you press a key, it creates

Device	What it does
	a **signal** (a small pattern of electricity) that the computer understands as an action, such as typing a letter or running a shortcut.
Mouse	A **pointing device**: moving it moves a **cursor** (the pointer) on the screen. It lets you **click**, **select**, and **drag** objects in a **graphical user interface (GUI)**, which is a screen with windows, icons, and buttons. Modern mice often have extra buttons and a **scroll wheel**.
Touchpad	A flat, **touch-sensitive surface**, usually found on laptops. Moving your finger across it moves the cursor, and tapping works like a click. Many touchpads also understand **gestures** (finger movements such as two-finger scrolling).
Camera	A **visual** input device that **captures images or video** and sends the data to the computer.
Microphone	Turns **sound waves into electric signals**, so audio can enter the computer.

Types of cameras:

- **Webcam**: A built-in or external camera on a computer, used for video calls.
- **Mobile camera**: Built into a smartphone or tablet. It has a **lens**, a **sensor** (which detects light), and an **image processor**, like a standalone camera but smaller.
- **Digital camera**: Higher quality, often used for photography and video production.

Types of microphones:

- **Condenser**: Very sensitive, often used in studios.
- **Dynamic**: Durable, often used for live sound.
- **USB microphone**: **Plug-and-play** (ready to use as soon as you connect it) and good for everyday use.

[insert image about a keyboard, a mouse, a touchpad, a webcam, and a microphone]

WORKED EXAMPLE

Task: You join a video call and type a message in the chat. Name the input devices used.

What you do	Input device
Show your face	Webcam (camera)
Speak	Microphone

What you do	Input device
Type the chat message	Keyboard
Move the pointer to the "Send" button on a laptop	Touchpad (or mouse)

ACTIVITY

Name the input device for each job:

a. Recording the sound of a live concert in a tough, durable way. b. Moving the cursor on a laptop by sliding your finger. c. Capturing a photo with your phone. d. Pressing `Ctrl + C` to copy text.

Answer

a. **Dynamic microphone** b. **Touchpad** c. **Mobile camera** d. **Keyboard**

2.2 Output Devices

Output devices present the **results of processed data** to the user. After the computer has processed the input, output devices share the results in different forms: **visual** (to see), **auditory** (to hear), or **physical** (to hold).

Device	What it does
Monitor	Also called a **display screen**. It shows data as graphics, text, or video.
Printer	Turns digital data into a **physical form** by printing on paper.
Speaker	Turns digital data into **sound**.

MONITOR

Monitors come in several types: **LCD**, **LED**, and **OLED**. They also differ in two measurements:

- **Resolution**: The number of tiny dots (**pixels**) that make up the picture. More pixels give a clearer picture.
- **Refresh rate**: How many times per second the screen **updates** the picture, measured in **hertz (Hz)**. A higher rate makes motion look smoother.

Monitors display the **desktop**, applications, and media, and they give feedback on what the user does. **High-definition** displays are important for graphic design, gaming, and video production, where accurate visuals matter.

PRINTER

Printers produce **hard copies** (paper copies) of documents. Common types:

- **Inkjet** and **laser** printers: Used for standard documents and photos.
- **3D printers**: Build **objects layer by layer** from a digital model. They are used in manufacturing, engineering, and medicine.

SPEAKER

Speakers **convert digital data into sound**, such as music, videos, alerts, and system notifications. They are important for multimedia and for **accessibility**, because they let visually impaired users get information through sound.

[insert image about a monitor, an inkjet printer, a 3D printer, and speakers]

WORKED EXAMPLE

Task: Choose the best output device.

Need	Device	Why
Watch a movie	Monitor	Shows video
Get a paper copy of a school form	Printer (inkjet or laser)	Makes a hard copy
Make a physical model of a part from a digital design	3D printer	Builds objects layer by layer
Hear a warning sound from the computer	Speaker	Produces audio

Refresh rate example: A monitor with a refresh rate of 60 Hz updates the picture 60 times every second. A 144 Hz monitor updates it 144 times every second, so motion looks smoother.

ACTIVITY

a. What is a pixel? b. How many times per second does a 120 Hz monitor update the picture? c. Which output device helps a visually impaired user hear information from the computer? d. What is a hard copy?

Answer

a. One of the **tiny dots** that make up the picture on a screen. b. **120 times** per second. c. **Speaker** d. A **paper copy** of a digital document or image.

2.3 Internal Hardware

MOTHERBOARD

The **motherboard** is the **main circuit board** of the computer. It is the **backbone**: it connects all the essential parts and lets them **communicate**, including the CPU, memory, storage, and input/output devices.

The motherboard has several important features:

Feature	What it does
CPU socket	The physical spot where the CPU is installed
Memory slots	Slots where RAM modules are plugged in
Expansion slots	Slots (such as **PCIe**) for adding extra cards, like a graphics card
Chipset	A group of chips that controls how the CPU, memory, and other devices work together
Data bus	The wires that carry data between the parts
Ports	Connectors for devices (see 2.4)

[insert image about a motherboard with the CPU socket, memory slots, expansion slots, and ports labeled]

THE DATA BUS

A **data bus** is a set of wires that moves data between the CPU, memory, and other parts. Its **width**, measured in **bits**, decides how much data can be sent **at the same time**. A wider bus can carry more data in one go. Some important buses are:

Bus	What it connects
Front-Side Bus (FSB)	The CPU and main memory (found in older designs)

Bus	What it connects
PCIe (Peripheral Component Interconnect Express)	High-speed parts such as graphics cards and SSDs
SATA	The motherboard and storage devices
USB buses	External devices such as keyboards and drives

The **bus specs** (specifications, or technical details) decide the **data transfer rate** and the system's performance.

THE CHIPSET

The **chipset** acts as a controller for the CPU, memory, and other devices. In many computers, it is split into two parts:

- **Northbridge**: Connects the CPU to **high-speed** parts such as RAM and graphics cards.
- **Southbridge**: Manages **lower-speed** parts such as hard drives, USB ports, and expansion slots.

(In many modern computers, the northbridge's work is built into the CPU itself, and a single chip handles the rest. The idea of fast and slow connections stays the same.)

[insert image about motherboard architecture: CPU, northbridge, southbridge, memory bus, and PCI bus]

WORKED EXAMPLE

Task: Compare data buses of different widths. How many transfers does it take to send 64 bits?

Bus width	Bits moved per transfer	Transfers needed for 64 bits
8 bits	8	64 ÷ 8 = **8 transfers**
32 bits	32	64 ÷ 32 = **2 transfers**
64 bits	64	64 ÷ 64 = **1 transfer**

A wider bus moves the same data in fewer steps.

ACTIVITY

a. What is the motherboard's main job? b. A 32-bit bus needs to send 128 bits. How many transfers are needed? c. Which part of the chipset connects the CPU to RAM and graphics cards?

Answer

a. To **connect all the parts** of the computer and let them communicate. b. 128 ÷ 32 = **4 transfers** c. The **northbridge**

CPU

The **Central Processing Unit (CPU)** is the **"brain"** of the computer. It **executes instructions** and **processes data**. It is installed in the **CPU socket** on the motherboard.

When the computer is switched on, the CPU starts the **boot sequence**. To **boot** means to start up. In the boot sequence, the CPU:

1. **Initializes** (sets up and checks) the hardware parts.
2. **Loads the operating system** (the main software that manages the computer) from storage into memory.

The CPU uses the data bus to **read and write data** to and from RAM and storage devices, which lets it work quickly.

[insert image about a CPU installed in a motherboard socket]

WORKED EXAMPLE

Task: Put the start-up steps of a computer in order.

1. You press the power button.
2. The CPU starts the boot sequence.
3. The hardware parts are initialized.
4. The operating system is loaded from storage into memory.
5. The computer is ready to use.

ACTIVITY

a. Where is the CPU installed? b. What does "boot" mean?

Answer

a. In the **CPU socket** on the motherboard. b. To **start up** the computer, setting up the hardware and loading the operating system.

RAM AND ROM

RAM (Random Access Memory) is the computer's fast, **temporary** working memory. When the CPU processes data, it keeps that data in RAM so it can reach it quickly. RAM is plugged into the **memory slots** on the motherboard. How much RAM there is, and how fast it is, affects the system's performance, especially for tasks that need fast access to data.

ROM (Read Only Memory) holds data that **cannot be easily changed**. It usually stores **firmware**, which is software built into the hardware to tell it how to start and work. For example, the **BIOS** is firmware that starts up a computer's hardware.

Feature	RAM	ROM
Full name	Random Access Memory	Read Only Memory
Purpose	Holds data and programs **in use right now**	Holds **start-up instructions** (firmware)
Can it be changed easily?	**Yes**, constantly	**No**, not easily
When the power goes off	Contents are **lost** (volatile)	Contents are **kept** (non-volatile)
Speed	Very fast	Slower than RAM

[insert image about RAM modules in memory slots and a ROM chip holding firmware]

WORKED EXAMPLE

Task: Decide whether RAM or ROM is used.

Situation	Memory	Why
A browser with 10 open tabs	RAM	It needs fast, changeable memory
The instructions a computer uses to start up	ROM	They must stay when the power is off
A game being played	RAM	The CPU needs the data quickly

ACTIVITY

a. Which memory is lost when the computer is switched off? b. What is firmware? c. Which memory is better for holding the start-up instructions, RAM or ROM?

Answer

a. **RAM** b. **Software built into hardware** that tells the hardware how to start and work. c. **ROM**

GRAPHICS CARD

A **graphics card** is an **expansion card** that handles **images and video**. It has its own processor, called a **GPU (Graphics Processing Unit)**, which is built to draw pictures very quickly. It plugs into a **PCIe expansion slot** on the motherboard and sends the picture to the monitor.

A **graphics card** is important for gaming, video editing, and graphic design.
Other **expansion cards** work the same way:

- **Sound card**: Improves audio for tasks like professional audio work.
- **Network card**: Connects the computer to a network.

Each expansion card has its own resources and **communicates with the CPU through the motherboard**. This lets users **customize** their computers for the performance they need.

[insert image about a graphics card plugged into a PCIe expansion slot]

WORKED EXAMPLE

Task: A designer edits 4K video. Which hardware helps most?

Part	Why it helps
Graphics card (GPU)	Draws images and video quickly
RAM	Holds the video data being edited
CPU	Runs the editing program

ACTIVITY

a. What does GPU stand for? b. Which slot is a graphics card plugged into? c. Name another type of expansion card.

Answer

a. **Graphics Processing Unit** b. A **PCIe expansion slot** on the motherboard c. A **sound card** (or **network card**)

POWER SUPPLY

The **power supply unit (PSU)** gives the computer the **electricity** it needs. The electricity from a wall outlet is **AC (alternating current)**, but computer parts need **DC (direct current)** at **lower voltages**. The PSU:

- **Converts** AC from the wall into low-voltage DC.
- **Sends power** through cables to the motherboard, drives, and other parts.

A PSU is rated in **watts (W)**. A more powerful computer, such as one with a strong graphics card, needs a PSU with a higher watt rating.

[insert image about a power supply unit with its cables going to the motherboard and drives]

WORKED EXAMPLE

Task: A computer's parts need about 350 W in total. Which PSU is suitable?

PSU	Suitable?	Why
250 W	No	Not enough power
500 W	Yes	Covers 350 W with room to spare

ACTIVITY

a. What does a power supply unit convert? b. If you add a powerful graphics card, why might you need a bigger PSU?

Answer

a. **AC** electricity from the wall into low-voltage **DC** for the computer's parts. b. A powerful graphics card uses **more electricity**, so the PSU must be able to supply more watts.

2.4 Ports and External Storage

Input/output ports are the **physical connectors** on a computer, such as **USB ports**, **audio jacks**, and **HDMI ports**. They let you connect devices to the computer. There are two types:

Type	What it connects	Examples
Internal port	The motherboard to devices **inside** the case	Hard disk, CD drive, internal Bluetooth
External port	The motherboard to devices **outside** the case	Mouse, printer, USB drive

[insert image about different port types: USB-A, USB-C, HDMI, audio jack, ethernet, DisplayPort]

USB (UNIVERSAL SERIAL BUS)

A **USB port** can connect many kinds of external devices: external hard disks, printers, scanners, mice, keyboards, and more. USB comes in different **shapes** (such as Type A, Type B, and Type C) and different **versions**, which have different **speeds**. Speed is measured in **megabits per second (Mbps)**, where one **megabit** is one million bits.

USB version	Maximum speed
USB 1.1 (an early version)	12 Mbps
USB 2.0	480 Mbps
USB 3.0	5,000 Mbps (5 Gbps)

A **USB flash drive** is a small, portable storage device that plugs into a USB port. It is plug-and-play and often used to move files between computers.

MEMORY CARD

A **memory card** is a small, thin **flash storage** card used to store data in cameras, phones, and other devices. A common type is the **SD card** (Secure Digital card), and a smaller version is the **microSD card**. You can read a memory card by putting it in a **card slot** or in a **card reader** (a small device that plugs into a USB port).

External storage means any storage that is not built permanently inside the computer, such as USB flash drives, external hard drives, and memory cards. It is useful for backups and for carrying files from one device to another.

[insert image about a USB flash drive and an SD memory card]

WORKED EXAMPLE

Task: How long does it take to copy a 600 MB file over each USB version? (A **byte** is 8 bits, so 1 megabyte (MB) = 8 megabits (Mb). These are the best-case speeds. Real speeds are usually a little lower.)

1. Convert the file size: 600 MB × 8 = **4,800 megabits**
2. Time at 12 Mbps (USB 1.1): 4,800 ÷ 12 = **400 seconds** (about 6.7 minutes)
3. Time at 480 Mbps (USB 2.0): 4,800 ÷ 480 = **10 seconds**

A newer USB version moves the same file much faster.

ACTIVITY

a. Name two examples of external ports. b. How long does it take to copy a 60 MB file at 480 Mbps? (Hint: 60 MB × 8 = 480 megabits.) c. What device can you use to read an SD card if your computer has no card slot? d. Give two examples of external storage.

Answer

a. For example, a **USB port** and an **HDMI port** (or an audio jack). b. 480 ÷ 480 = **1 second** c. A **card reader** that plugs into a USB port. d. For example, a **USB flash drive** and a **memory card** (or an external hard drive).

STAGE 2 SUMMARY

- **Hardware** is the physical part of a computer. **Internal hardware** is inside the case, and **external hardware** (peripherals) connects from outside.
- **Input devices** send data into the computer: **keyboard**, **mouse**, **touchpad**, **camera**, and **microphone**.
- **Output devices** present results: **monitor** (resolution and refresh rate), **printer** (inkjet, laser, 3D), and **speaker**.
- The **motherboard** connects all the parts through the **data bus**, the **chipset**, **memory slots**, **expansion slots**, and **ports**.
- The **CPU** is the brain. It starts the boot sequence and uses the data bus to work with RAM and storage.
- **RAM** is fast, temporary working memory, and **ROM** keeps firmware that starts the hardware.
- A **graphics card (GPU)** handles images and video, and the **power supply** converts wall AC into low-voltage DC for the computer.
- **Ports** connect devices. **USB** connects many kinds of devices, and **memory cards** and USB drives provide portable **external storage**.

Stage 3: Motherboard Power and BIOS, CPU, and GPU

LEARNING GOALS

By the end of this stage, you will be able to:

1. Identify core physical components of computers and explain basic functions.
2. Explain how the motherboard shares power and how the BIOS/UEFI starts the computer.
3. Describe the CPU and the fetch-decode-execute cycle.
4. Explain what decides a CPU's speed: clock speed, cache, bus speed, and threads.
5. Describe the GPU, how it works, and why it is useful for graphics, gaming, and AI.

Topics covered (from Internal Hardware):

- Motherboard (power distribution and BIOS/UEFI)
- CPU
- Graphics Card (GPU)

3.1 Internal Hardware

Hardware means the physical parts of a computer. **Internal hardware** means the parts that sit **inside** the computer case. The **motherboard** is the main circuit board that connects them all, so it is a good place to start.

MOTHERBOARD: POWER DISTRIBUTION AND BIOS/UEFI

POWER DISTRIBUTION

The **power supply unit (PSU)** brings electricity into the computer.
The **motherboard** then **shares (distributes) this power** with the other parts, including the **CPU**, **memory**, and **peripheral devices** (devices that connect to the computer, like a mouse or printer).

Each part needs a different amount of power. The motherboard has **voltage regulation circuits**. **Voltage** is the "push" of electricity, measured in **volts**. These circuits make sure **each part receives a steady, correct level of voltage**. This is very important for **reliability** (the computer keeps working without crashing) and for **performance**.

BIOS/UEFI

The motherboard also holds a special piece of **firmware**. **Firmware** is software stored permanently inside a hardware device to tell it how to work. This firmware is called:

- **BIOS (Basic Input/Output System)**: The older name.
- **UEFI (Unified Extensible Firmware Interface)**: The newer version that replaces BIOS in most computers.

The BIOS/UEFI has two main jobs:

1. **It starts up the hardware** during the **boot process**. (To **boot** means to start the computer.) It **initializes** the hardware, which means it sets up and checks each part.
2. **It gives you a low-level way to configure the system.** (**Low-level** means close to the hardware, before the normal operating system has even started.) It stores important **settings** for hardware parts, and lets you change the **boot order**, which is the order in which the computer looks for an operating system, and some **hardware performance options**.

[insert image about the motherboard sharing power from the PSU to the CPU, memory, and peripherals]

[insert image about a BIOS/UEFI settings screen showing boot order]

WORKED EXAMPLE

Task: A computer has a USB drive plugged in, and you want to start it from the drive instead of the SSD.

Step	What happens
1	You turn on the computer and open the BIOS/UEFI settings
2	You change the **boot order** so the USB drive is first and the SSD is second
3	You save and restart
4	The BIOS/UEFI initializes the hardware, then looks for an operating system on the USB drive first

ACTIVITY

a. What does the motherboard do with the electricity it receives from the PSU? b. Why does the motherboard have voltage regulation circuits? c. What do BIOS and UEFI stand for? d. Name two things you can change in the BIOS/UEFI.

Answer

a. It **distributes** the power to the CPU, memory, and peripheral devices. b. To make sure **each part gets a stable, correct power level**, which keeps the computer reliable and working well. c. **Basic Input/Output System** and **Unified Extensible Firmware Interface**. d. For example: the **boot order**, **hardware settings**, and **performance options**.

CENTRAL PROCESSING UNIT (CPU)

The **Central Processing Unit (CPU)** is the **"brain"** of the computer. It **executes instructions** (carries out the steps of programs) and **processes data**. It sits in the **CPU socket** on the motherboard.

When the computer is switched on, the CPU begins the **boot sequence**: it **initializes the hardware** and **loads the operating system** from storage into memory.

Inside the CPU are **millions or even billions of transistors**. A **transistor** is a tiny electronic **switch** that is either on or off. This matches **binary** (the system of `0`s and `1`s), so the CPU receives all its data in binary.

THE FETCH-DECODE-EXECUTE CYCLE

After the CPU receives input, it follows three steps for every instruction:

Step	What happens
1. Fetch	The CPU gets the instruction from **RAM**. It uses the **address bus** (a set of wires that carries a memory **address**, which is a location number) to find where the instruction is stored. The instruction is then loaded into the CPU.
2. Decode	The CPU works out **what the instruction means**, by breaking it into smaller parts it can carry out.
3. Execute	The CPU carries out the instruction using its **ALU**. The result is stored back in RAM or sent to an **output device**, as needed.

This cycle repeats in a **continuous loop**, which lets the CPU do complicated tasks quickly.

[insert image about the fetch-decode-execute cycle between the CPU, main memory, and the I/O system]

PARTS OF THE CPU

- **Control unit**: Directs how the processor works. It manages the flow of data between the CPU and other parts by **sending signals** that coordinate actions, such as reading from or writing to memory.
- **Arithmetic Logic Unit (ALU)**: The part that does the **actual computing**. It does **arithmetic operations** (add, subtract, multiply, divide) and **logic operations** (comparisons, and the operations AND, OR, and NOT). It takes its inputs from the CPU's **registers**, which are tiny, very fast storage spots inside the CPU, and then outputs the result.
- **Program counter**: A register that holds the **address of the next instruction** to fetch.

- **Instruction register (IR)**: A register that holds the **instruction currently being decoded or executed**.

[insert image about the inside of a CPU: program counter, registers, ALU, control unit, and IR]

WORKED EXAMPLE

Task 1: Follow one instruction through the cycle. The instruction "Add 4 and 6" is stored at memory address 100.

Step	What happens
Fetch	The program counter points to address 100. The CPU uses the address bus to get the instruction from RAM.
Decode	The control unit works out that it means "add".
Execute	The ALU calculates 4 + 6 = 10. The result is stored back in RAM.
Next	The program counter moves to the next address, and the cycle repeats.

Task 2 (clock speed, simplified): A CPU runs at 3 GHz, and each instruction takes 3 cycles.

1. 3 GHz = 3 billion cycles per second
2. Instructions per second = 3 billion ÷ 3 = **1 billion instructions per second**

ACTIVITY

a. Put these steps in order: **Execute**, **Fetch**, **Decode**. b. What is the ALU's job? c. A CPU runs at 4 GHz. How many cycles does it perform each second? d. Which cache is the fastest: L1, L2, or L3? e. In the CPU-Z example, how many threads can the Intel Core i5-12500H handle?

Answer

a. **Fetch → Decode → Execute** b. To do the **arithmetic** (math) and **logic** (comparison) operations. c. 4 GHz = **4 billion** cycles per second. d. **L1** e. **16 threads**

GRAPHICS PROCESSING UNIT (GPU)

A **Graphics Processing Unit (GPU)** is a **specialized processor** made to handle complex calculations, mainly those used to **draw (render) visuals**, **run games**, and do tasks that need **parallel processing**, such as AI. (**Rendering** means turning data into a finished picture. **Parallel processing** means doing many tasks **at the same time**.)

The GPU works **alongside the CPU**, but it is built to perform **many repetitive calculations at once**. That makes it perfect for graphics-heavy work.

	CPU	GPU
Style	A few powerful cores that handle varied tasks one after another	Many small cores that handle many simple tasks at the same time
Best at	General tasks, running the operating system and programs	Graphics, video, games, and AI calculations

CORE COMPONENTS OF A GPU

Component	What it does
Cores	Small processing units that run tasks **in parallel**. More cores usually mean better performance for many tasks at once.
Memory (VRAM)	Video RAM. Stores data needed for drawing, such as **textures** (surface pictures on objects) and **frame buffers**. It lets the GPU work fast without always using the system's RAM.
Memory interface	Controls the flow of data between the VRAM and the GPU cores. It affects how fast the GPU can reach stored data. Common memory types are **GDDR** (Graphics Double Data Rate), such as **GDDR6** or **GDDR6X**, built for high-speed transfers.
Shaders	Specialized units that handle **lighting, color**, and other visual effects.
Rasterizer	Turns **3D objects into pixels** to show on a flat, 2D screen. (A **pixel** is one tiny dot of a picture.)
Frame buffer	Holds the **finished image** before it is sent to the display.
Control logic	Directs the cores and **schedules** (organizes) their tasks.

[insert image about the main parts inside a GPU]

HOW A GPU WORKS

1. **Input data**: The CPU sends the GPU data, such as textures, **vertices** (corner points of 3D shapes), and shaders.
2. **Parallel processing**: The GPU's many cores split the data into small tasks and work on them **at the same time**.
3. **Rendering**: The tasks include calculating images, textures, and animations.

4. **Output**: The finished data is sent to the **display**, producing images or video.

WHAT GPUS ARE USED FOR

- **Complex visuals and rendering**: GPUs draw 2D and 3D graphics, animations, and video. In video editing, **CAD** (Computer-Aided Design, used to design objects), and animation, the GPU handles shading, textures, and polygon creation much faster than a CPU.
- **Gaming**: Modern games need **high frame rates** (many images per second), realistic lighting, shadows, and physics. GPUs make smooth, lifelike visuals possible, including in **virtual reality (VR)**.
- **Artificial intelligence and machine learning**: AI training uses a huge number of **matrix multiplications** (repeated multiplying of grids of numbers), which a GPU can do in parallel. Companies such as NVIDIA build specialized GPUs for AI.

INTEGRATED GPUS

Some GPUs are **integrated**, which means they are **built into the CPU** instead of being a separate graphics card. Integrated GPUs are:

- **Energy-efficient**, so they suit **battery-powered devices** such as laptops.
- **Cost-effective**, because there is no need to buy a separate graphics card.

A **dedicated** GPU is a separate graphics card with its own VRAM. It is more powerful but costs more and uses more power.

WHAT DECIDES A GPU'S SPEED?

Factor	Meaning
Core clock	The speed at which the GPU cores run, in **MHz** (megahertz). It directly affects processing power.
Memory clock	The speed of the VRAM, in MHz. It shows how fast data can move between the memory and the GPU.

[insert image about a CPU sending data to a GPU, which sends the final image to a display]

WORKED EXAMPLE

Task 1 (pixels per second): A monitor shows 1920 × 1080 pixels at 60 frames per second. How many pixel updates does the GPU handle every second?

1. Pixels per frame = 1920 × 1080 = **2,073,600**
2. Pixels per second = 2,073,600 × 60 = **124,416,000**

That is more than 124 million pixel updates every second, which is why a GPU's parallel cores are so helpful.

Task 2 (parallel processing): There are 100 tasks, and each takes 1 second.

Number of cores	Time needed
1 core	100 seconds
10 cores	10 seconds
100 cores	1 second

More cores working in parallel finish the work faster.

ACTIVITY

a. What does GPU stand for? b. What is parallel processing? c. How many pixels are on a 3840 × 2160 (4K) screen? d. Name two parts inside a GPU and what each one does. e. Why is an integrated GPU a good choice for a laptop?

Answer

a. **Graphics Processing Unit** b. Doing **many tasks at the same time**. c. 3840 × 2160 = **8,294,400 pixels** d. For example: **VRAM** stores textures and frame buffers; the **rasterizer** turns 3D objects into pixels; the **frame buffer** holds the finished image. e. It is **energy-efficient**, which helps battery life, and **cost-effective**, because no separate graphics card is needed.

STAGE 3 SUMMARY

- The **motherboard** distributes power from the PSU to the CPU, memory, and peripherals, and uses **voltage regulation circuits** to keep each part's power steady.
- The **BIOS/UEFI** is firmware that **initializes the hardware** during boot and lets you change settings, such as the **boot order**.
- The **CPU** is the "brain". It is built from billions of **transistors** and repeats the **fetch-decode-execute** loop.
- The **control unit** directs the CPU's work, and the **ALU** does arithmetic and logic.
- CPU speed depends on **clock speed**, **cache** (L1, L2, L3), **bus speed**, and **threads**.
- The **GPU** is a specialized processor that uses **parallel processing** for graphics, gaming, and AI.
- A GPU's main parts are **cores, VRAM, a memory interface, shaders, a rasterizer, a frame buffer, and control logic**.
- GPU speed depends on the **core clock** and the **memory clock**. **Integrated GPUs** are energy-efficient and low in cost, while **dedicated** GPUs are more powerful.

Stage 4: Graphics Memory, RAM, ROM, and Storage

LEARNING GOALS

By the end of this stage, you will be able to:

1. Identify core physical components of computers and explain basic functions.
2. Explain what VRAM is and read the main details of a graphics card.
3. Explain the difference between RAM and ROM, and where cache memory fits.
4. Describe what decides RAM performance: frequency, type, and timings.
5. Describe secondary storage devices (HDD, SSD, optical, USB flash drives, memory cards) and their strengths and weaknesses.
6. Explain how the CPU retrieves data using the fetch-decode-execute cycle and interrupts.

Topics covered (from Internal Hardware and Ports and External Storage):

- Graphics Card (VRAM)
- RAM
- RAM vs ROM
- USB and external storage (flash drives, memory cards)

4.1 Graphics Card: VRAM and Specs

A graphics card has its own fast memory called **VRAM** (Video RAM). It stores the data needed to draw pictures, such as textures and the final image before it is shown on screen.

VRAM comes in different **types**. Each new type is usually **faster at transferring data** than the one before it. For example, **GDDR6** is faster than **GDDR5**. (**GDDR** stands for Graphics Double Data Rate.) Newer types are common in modern graphics cards used for demanding jobs, such as gaming and video editing.

A **dedicated GPU** is a graphics card with its own VRAM. Here is a real example. A laptop with an **NVIDIA RTX 3050** graphics card shows these specs in the program **CPU-Z**:

Detail	Value	What it means
Name	NVIDIA GeForce RTX 3050 Laptop GPU	The graphics chip's model
Code name	GA107	The name of the chip's design

Detail	Value	What it means
Technology	8 nm	The size of the tiny parts inside the chip. A **nanometer (nm)** is one billionth of a meter. Smaller sizes usually mean more parts fit on the chip and it uses less power.
Max TDP	80 W	The most power (and heat) the chip is designed to handle, in watts
GFX core clock	210 MHz	The current speed of the GPU core (it changes depending on the workload)
Shader clock	405 MHz	The current speed of the shaders, the units that handle lighting and color
Memory size	4 GB	How much VRAM there is
Memory type	GDDR6	The type of VRAM
Bus width	128 bits	How many bits of data can move between the VRAM and the GPU at the same time

[insert image about the CPU-Z program showing the specs of an NVIDIA GeForce RTX 3050 Laptop GPU]

WORKED EXAMPLE

Task: The RTX 3050 has a **128-bit** memory bus. How many bytes can it move in one transfer, and how many transfers are needed to move 1,600 bytes?

1. 1 byte = 8 bits, so 128 bits ÷ 8 = **16 bytes per transfer**
2. 1,600 bytes ÷ 16 = **100 transfers**

A wider bus moves the same data in fewer transfers.

ACTIVITY

a. What does VRAM store? b. Which is the newer and faster memory type: GDDR5 or GDDR6? c. How much VRAM does the RTX 3050 in the example have? d. A memory bus is 256 bits wide. How many bytes can it move in one transfer?

Answer

a. Data needed for drawing images, such as **textures** and the **frame buffer**. b. **GDDR6** c. **4 GB** d. 256 ÷ 8 = **32 bytes**

4.2 Storage

PRIMARY STORAGE: RAM, ROM, AND CACHE

Storage is the part of the computer that holds data the CPU and other components need to reach **right away** while the computer is working. There are two main groups: **primary storage** and **secondary storage**. We start with primary storage.

RAM (Random Access Memory) keeps data **temporarily** while the computer is running. RAM gives **very quick access** to data, but **everything in it is lost when the power goes off**. (This is called being **volatile**.) RAM acts as a **high-speed buffer**, which means a fast waiting area, between the CPU and slower storage devices such as hard drives. This lets the computer quickly reach the data it needs to run programs and carry out commands.

ROM (Read Only Memory) keeps data that **cannot be easily changed**. In modern computers, ROM usually holds **firmware**, which is software stored inside a hardware device to tell it how to work.

Feature	RAM	ROM
Full name	Random Access Memory	Read Only Memory
Holds	Programs and data in use right now	Firmware (start-up instructions)
Can it be changed easily?	Yes	No
When the power goes off	Contents are lost	Contents are kept

[insert image about a RAM stick and a ROM chip, labeled RAM and ROM]

Where RAM sits in the computer:

- The **motherboard** has **memory slots** for **DRAM modules**. (**DRAM** is Dynamic RAM, the common type of RAM. A **module** is a small circuit board holding the memory chips.)
- Common module types are **DDR4** and **DDR5**. They have different speeds and sizes.
- How well the memory talks to the CPU also depends on the motherboard's **chipset** and **bus speed**.

Cache memory: In modern computers, the CPU usually has **cache memory**. It is a **smaller and faster** type of volatile memory **very close to the CPU**, and it keeps the most frequently used data so the CPU can reach it even faster than RAM.

WORKED EXAMPLE

Task: Decide where each item is kept.

Item	Where	Why
A game while you are playing it	RAM	Needs fast access, and is only needed while it runs
The instructions a computer uses to start up	ROM	They must stay when the power is off
The data the CPU uses again and again	Cache	Fastest access, right next to the CPU

ACTIVITY

a. Which memory loses its contents when the computer is switched off, RAM or ROM? b. What does "buffer" mean when we say RAM is a high-speed buffer? c. Name two RAM module types. d. Where is cache memory located?

Answer

a. **RAM** b. A **fast waiting area** between the CPU and slower storage, where data is held for quick access. c. **DDR4** and **DDR5** d. **Very close to the CPU** (inside or right next to it).

RAM PERFORMANCE

RAM performance is usually described using three things:

Term	Meaning
RAM frequency	How fast the RAM can **read and write** data, measured in **MHz** (megahertz, millions of cycles per second), for example 3200 MHz. A higher frequency means faster data transfer between the RAM and the CPU.
RAM type	The technology **generation**, such as DDR4 or DDR5. Newer types are usually faster and more efficient.

Term	Meaning
RAM timings	Numbers that show **latency**, which is the **delay** before data starts to arrive. **Lower numbers mean faster performance.**

Here are the main details of a real RAM module, as shown in **CPU-Z**:

Detail	Value	What it means
Type	DDR4	The RAM generation
Size	8 GB	The amount of memory
Channel	1 × 64-bit	How data travels to the CPU: one path that is 64 bits wide
DRAM frequency	1596.1 MHz	The speed of the RAM's clock
CAS latency (CL)	22 clocks	The delay, counted in clock cycles, before data can be read. This is the best-known timing number.
RCD, RP, RAS, RC	22, 22, 52, 74 clocks	Other timing numbers for opening and closing parts of the memory
Command rate	1T	How quickly the memory accepts commands

(DDR memory sends data twice per clock cycle. That is why a clock of about 1600 MHz is sold as "3200" memory.)

[insert image about the CPU-Z program showing the specs of a DDR4 RAM module]

WORKED EXAMPLE

Task: Which RAM responds faster? Both modules run at 1596.1 MHz. Module A has CL = 22, and Module B has CL = 16.

1. One clock cycle lasts 1 ÷ 1596.1 million seconds, which is about **0.627 nanoseconds** (a **nanosecond** is one billionth of a second).
2. Module A: 22 × 0.627 ≈ **13.8 ns**
3. Module B: 16 × 0.627 ≈ **10.0 ns**

Module B has a **lower timing number**, so it has less delay and is **faster**.

ACTIVITY

a. What does a lower timing number mean? b. Which gives faster transfer: 2666 MHz or 3200 MHz RAM (same type)? c. How much RAM is in the CPU-Z example? d. Name the two RAM generations mentioned in this section.

Answer

a. **Less delay**, so faster performance. b. **3200 MHz** (a higher frequency means faster transfer). c. **8 GB** d. **DDR4** and **DDR5**

SECONDARY STORAGE AND USB DRIVES

Secondary storage is **permanent** storage. It includes hard drives (HDD), solid-state drives (SSD), optical drives, and USB flash drives. Data **stays stored** here even when the computer is turned off.

Device	How it works	Notes
Hard Disk Drive (HDD)	A traditional drive that stores large amounts of data on **spinning magnetic disks** called **platters**	Big capacity, slower, has moving parts
Solid-State Drive (SSD)	**Flash-based** storage (it uses electronic memory chips) with **no moving parts**	Much faster than an HDD
Optical storage	Discs such as **CDs, DVDs, and Blu-rays**, read by a laser	Used for media and backup copies
USB flash drive	A **portable, solid-state** storage device that plugs into a **USB port**	Used to move files between devices and for backups. Capacity ranges from a few gigabytes to several terabytes.

A **memory card**, such as an **SD card**, is another small, portable flash storage device. It is commonly used in cameras and phones.

Advantages of USB drives:

- **Portability**: Small and light, so they are easy to carry.
- **Plug-and-play**: No installation is needed. You plug the drive into a USB port and it is ready to use.

Disadvantages of USB drives:

- **Limited lifespan**: Flash storage can only be written a limited number of times (called **write cycles**). For most people this does not matter unless the drive is used very heavily.
- **Easy to lose or damage**: Because they are so small, they can be misplaced or damaged if not handled carefully.

[insert image about different storage media: a floppy disk, USB drives, a CD, a hard drive, and an SD card]

WORKED EXAMPLE

Task: Choose the best storage for each need.

Need	Best choice	Why
Carry a school project to another computer	USB flash drive	Portable and plug-and-play
Make a computer start and open programs quickly	SSD	Fast, with no moving parts
Store a large archive of old files cheaply	HDD	Large capacity
Keep the photos from a camera	Memory card (SD card)	Small, made for cameras

ACTIVITY

a. Which type of drive has spinning platters? b. Give one advantage and one disadvantage of USB drives. c. What is a write cycle?

Answer

a. The **HDD** (hard disk drive). b. Advantage: for example, **portability** or **plug-and-play**. Disadvantage: for example, a **limited lifespan** or **easy to lose or damage**. c. **One time data is written** to the flash storage. Flash storage can only handle a limited number of them.

DATA ACCESS AND RETRIEVAL

Primary and secondary storage work together:

- **Primary storage (RAM)** is **much faster** than secondary storage, so the CPU can reach data quickly while it works. But because it is **volatile**, it needs a **constant power supply**. That is why data that is not in active use is **moved to secondary storage**.

- **Secondary storage** has **much more space** and keeps data for the long term, but it has **slower access**. It is used for saving documents, system files, applications, and **databases** (organized collections of data).

How the CPU retrieves data: the fetch-decode-execute cycle

Step	What happens
Fetch	The CPU gets the instruction or data from memory
Decode	The CPU works out what action the instruction asks for
Execute	The CPU does the action, such as adding numbers or moving data

Input/Output (I/O) and interrupts

I/O devices (like a keyboard or printer) send and receive data using **interrupts**.
An **interrupt** is a **signal that pauses** the CPU's normal work so it can **handle an input or output task**. After that, the CPU **returns to the cycle** where it left off.

[insert image about the CPU pausing the fetch-decode-execute cycle to handle an interrupt from a keyboard]

WORKED EXAMPLE

Task: You are running a program, and you press a key.

Step	What happens
1	The CPU is in the middle of the fetch-decode-execute cycle for your program
2	The keyboard sends an **interrupt** signal
3	The CPU **pauses** the program and handles the key press (the input task)
4	The CPU **returns** to the cycle and carries on with the program

ACTIVITY

a. Why is data not in use moved from RAM to secondary storage? b. Which has more storage space, primary or secondary storage? c. What does an interrupt do? d. Put in order: **Execute**, **Fetch**, **Decode**.

Answer

a. RAM is **volatile** and needs constant power, so data that must be kept is moved to **secondary storage**, which keeps it without power. b. **Secondary storage** c. It **pauses the CPU's normal processing** so it can handle an input or output task, then the CPU returns to the cycle. d. **Fetch → Decode → Execute**

STAGE 4 SUMMARY

- **VRAM** is the graphics card's own memory. Newer types (like **GDDR6**) are faster than older ones (like GDDR5). A **128-bit** bus moves 16 bytes per transfer.
- **RAM** is fast, temporary memory that loses its contents without power. **ROM** keeps **firmware** that cannot be easily changed. **Cache** is small, very fast memory near the CPU.
- **RAM performance** depends on **frequency** (MHz), **type** (DDR4 or DDR5), and **timings** (lower means less delay).
- **Secondary storage** keeps data without power: **HDD** (spinning platters), **SSD** (flash, no moving parts), **optical discs**, **USB flash drives**, and **memory cards**.
- **USB drives** are portable and plug-and-play, but they have a limited number of write cycles and are easy to lose or damage.
- **RAM** is faster and **secondary storage** is larger and permanent, so data not in use is moved to secondary storage.
- The CPU retrieves data using **fetch-decode-execute**. **Interrupts** pause that cycle so the CPU can handle input and output.



Learning Goals

- 1) Aptos / Aptos Display / explain its basic worldtiow (Input, Processing, Storage, Output).
- 2) Convert between binary, decimal, and hexadecimal number systems.
- 3) Explain how different data types (numbers, text, images, audlo, video) are stored as binary data.
- 4) Distinguish between primary storage (RAM) and secondary storage (SSD/HDD).
- 5) Describe the Von Neumann model and how internal hardware components (CPU, ALU, Memory) work together to execute a program.
- 6) Identify core hardware components and distinguish between internal hardware and external peripherals.
- 7) Define a computer network and distinguish between different network types (LAN vs. WAN).
- 8) Explain how the intemet delivers data using TCP/IP, IP addresses, and domain names (DNS).
- 9) Identify the four core network security goals (Privacy, Authentication, Integrity, Nonrepudiation) and match common protections to each. Color

Chapter 3: Network and Internet Basics

Stage 1: What Is a Computer Network?

LEARNING GOALS

By the end of this stage, you will be able to:

1. Describe how data travels between connected devices using networks.
2. Define a computer network and explain how one works.
3. Explain the terms IP address, hostname, DNS, protocol, and bandwidth.
4. Describe the client-server and peer-to-peer network designs.
5. Give reasons why computers are networked.

Topics covered:

- Network overview
- Why computers are networked

1.1 Network Overview

WHAT IS A COMPUTER NETWORK?

A **computer network** is a group of **connected devices** that **share data and resources** with each other.

Let's define the words in that sentence:

- **Node**: Any device that is connected to a network, such as a laptop, a phone, a printer, or a server.
- **Resource**: Something that can be shared, such as a printer, a file, or an internet connection.
- **Server**: A computer that provides data or services to other devices on a network.

Devices can be connected in two ways:

Connection	Meaning	Example
Wired	Uses **cables**	An Ethernet cable between a laptop and a router

Connection	Meaning	Example
Wireless	Uses **radio waves** instead of cables	Wi-Fi

Once devices are connected, they follow **protocols** to exchange data. A **protocol** is a set of agreed rules that devices follow so that they can understand each other, like two people agreeing to speak the same language. Examples of protocols:

Protocol	Full name	What it is used for
TCP/IP	Transmission Control Protocol / Internet Protocol	The main rules for sending data across networks and the Internet
SMTP	Simple Mail Transfer Protocol	Sending email
HTTP	Hypertext Transfer Protocol	Loading web pages

[insert image about several nodes (laptop, phone, printer, server) connected in a network]

WORKED EXAMPLE

Task: Two laptops are connected with an Ethernet cable, and one sends a file to the other. Identify the parts of the network.

Part	In this example
Nodes	The two laptops
Connection	A wired connection (the Ethernet cable)
Resource shared	The file
Protocol	Rules such as TCP/IP that control how the file is sent

ACTIVITY

a. What is a node? b. Name one wired and one wireless way of connecting devices. c. What does HTTP help a computer do?

Answer

a. Any **device connected to a network**, such as a laptop, a phone, or a printer. b. Wired: an **Ethernet cable**. Wireless: **Wi-Fi**. c. **Load web pages.**

THE HISTORY AND SIZE OF NETWORKS

The first example of a computer network was the **ARPANET** (Advanced Research Projects Agency Network). It was created in the **late 1960s** by **ARPA**, an agency of the U.S. Department of Defense. It was a **packet-switched** network, which means that data was broken into small pieces called **packets**, and each packet was sent separately and put back together at the other end.

[insert image about the early ARPANET with its nodes at universities and research centers]

Networks come in many sizes:

- **Very small**: Two laptops joined by an Ethernet cable.
- **Very large**: The **Internet**, which is a **global system of connected computer networks**.

Two common network sizes:

Type	Full name	Area it covers	Example
LAN	Local Area Network	A small area	A home, a school, or an office
WAN	Wide Area Network	A large area, such as cities or countries	The Internet

WORKED EXAMPLE

Task: Decide whether each network is a LAN or a WAN.

Network	Type	Why
The computers in one classroom	LAN	Small area
A company linking its offices in three countries	WAN	Covers a large area
The Internet	WAN	Covers the whole world

ACTIVITY

a. Which network was the first example of a computer network? b. What is a packet? c. Is a home Wi-Fi network a LAN or a WAN?

Answer

a. The **ARPANET**, created in the late 1960s. b. A **small piece of data** that is sent across a network and put back together at the destination. c. A **LAN**.

HOW DOES A COMPUTER NETWORK WORK?

A network works through a few simple ideas:

1. Devices use IP addresses.

An **IP address** (Internet Protocol address) is a **unique number** that identifies a device on a network, like a house address for a letter. An example looks like `192.168.1.25`.

2. Hostnames and DNS make addresses easy to use.

People find numbers hard to remember, so websites also have **hostnames**, which are names such as `www.example.com`. The **Domain Name System (DNS)** is a system of **DNS servers** that works like a phone book: it **looks up a hostname and gives back the matching IP address**, so devices can find each other.

3. Protocols and algorithms control how data is sent.

An **algorithm** is a step-by-step set of rules for doing a task. Protocols and algorithms together decide **how data is split up, sent, and put back together**.

4. Networks follow standards.

A **standard** is an agreed technical rule that lets equipment from different makers work together. Standards are set by organizations such as the **IEEE** (Institute of Electrical and Electronics Engineers):

Standard	Used for
Ethernet	Wired networks
802.11	Wireless networks (Wi-Fi)

[insert image about a DNS server turning a hostname into an IP address]

WORKED EXAMPLE

Task: Follow what happens when you open a website by typing its name.

Step	What happens
1	You type the hostname `www.example.com` into your browser

Step	What happens
2	Your device asks a **DNS server**, "What is the IP address for this name?"
3	The DNS server replies with the **IP address** (for example, `192.0.2.10`)
4	Your device sends a request to that IP address using **HTTP** over **TCP/IP**
5	The server sends the web page back, as packets
6	Your browser puts the packets together and shows the page

ACTIVITY

a. What does DNS do? b. What is the difference between a hostname and an IP address? c. Which standard is used for Wi-Fi? d. Who sets networking standards such as Ethernet and 802.11?

Answer

a. It **turns a hostname into an IP address**, so devices can find each other. b. A **hostname** is a name that people can read (like `www.example.com`). An **IP address** is the number that devices use to find each other (like `192.0.2.10`). c. **802.11** d. Organizations such as the **IEEE**.

NETWORK DESIGN (ARCHITECTURE) AND BANDWIDTH

Network architecture is the **physical and logical design** of a network, or how its parts are arranged and how they work together. There are two common designs:

Design	How it works	Example
Client-server	**Clients** (devices that ask for things) request resources from a central **server**	A phone (client) asks a website's server for a web page
Peer-to-peer (P2P)	Every device is an equal, and the devices **share resources** like memory and processing power directly with each other	Friends sharing files directly between their computers

[insert image about a client-server network next to a peer-to-peer network]

Capacity, also called **bandwidth**, measures **how much traffic (data) a network can handle** while still performing well. It is measured in **bits per second** (for example, **Mbps**, or

megabits per second, where 1 megabit is one million bits). A higher bandwidth means more data can move at once.

WORKED EXAMPLE

Task 1: A school computer asks the school's file server for a document. Which design is this?

Client-server. The computer is the client, and the file server is the central server that provides the document.

Task 2: How long does it take to send a 50 MB file over a 100 Mbps connection? (1 byte = 8 bits, so 1 MB = 8 megabits.)

1. File size in megabits: 50 × 8 = **400 megabits**
2. Time = 400 ÷ 100 = **4 seconds** (in the best case; real speeds are usually a little slower)

ACTIVITY

a. In a client-server network, what does the client do? b. In which design do devices share resources directly with each other? c. How long does a 25 MB file take to send over a 100 Mbps connection? (Hint: 25 × 8 = 200 megabits.)

Answer

a. It **requests resources** from the server. b. **Peer-to-peer (P2P)**. c. 200 ÷ 100 = **2 seconds**.

1.2 Why Computers Are Networked

Computer networks make it possible to **share information quickly** and to **use resources efficiently**. Here are the main reasons why computers are networked:

Benefit	What it means	Example
Resource sharing	Many devices share the same resources, such as printers, files, scanners, and photocopiers. This is helpful for large organizations that are spread out around the world, because they can use one network to connect all their employees.	An office where 20 people print on one shared printer
Flexibility	People can choose the communication and sharing method that suits them.	Using email, instant messaging, or an app like WhatsApp

Benefit	What it means	Example
Higher connectivity	People can stay connected **wherever they are**. Video calling and document-sharing apps let workers **collaborate** (work together) from a distance.	Zoom meetings and Google Docs
Data security and management	Data is **centralized** (kept in one main place) on shared servers. **Network administrators** (the people who run the network) can manage and protect the data more easily, make regular **backups** (extra copies), and apply security measures to all devices at once, such as **multi-factor authentication** (checking identity with more than one proof, like a password plus a phone code).	A company backing up all its files every night from one server
Storage capacity	Organizations grow and have more and more data. Networks, especially ones using **cloud technology** (storage and services provided over the Internet), can store huge amounts of data and backups on a central remote server that everyone can reach at any time.	Saving files to a cloud drive
Entertainment	Networks, especially the Internet, provide games, streaming music, and videos. Multiplayer games need a **LAN** or a **WAN** (such as the Internet) to work.	Playing an online game with friends

[insert image about the benefits of networks: sharing a printer, video calls, cloud storage, and online gaming]

WORKED EXAMPLE

Task 1: An office has 10 employees who all need to print. A printer costs $300. Compare buying one printer for each person with sharing one printer over a network.

Option	Number of printers	Total cost
No network (one printer each)	10	10 × $300 = **$3,000**
Network (one shared printer)	1	**$300**

The network saves **$2,700**. This is the benefit of **resource sharing**.

Task 2: Match each situation to a benefit of networking.

Situation	Benefit
A team edits the same document from different cities	Higher connectivity
A company backs up everything from one server	Data security and management
A student saves a large project in the cloud	Storage capacity
Friends play an online match together	Entertainment

ACTIVITY

Name the benefit of networking in each case:

a. A school has one scanner that all teachers use. b. An IT manager turns on multi-factor authentication for every computer at once. c. A worker joins a video call from home. d. A person can choose to message a friend with email or with WhatsApp.

Answer

a. **Resource sharing** b. **Data security and management** c. **Higher connectivity** d. **Flexibility**

STAGE 1 SUMMARY

- A **computer network** is a group of connected **nodes** (devices) that share data and resources, using **wired** or **wireless** connections and **protocols** such as TCP/IP, SMTP, and HTTP.
- The **ARPANET** (late 1960s) was the first example of a computer network. The **Internet** is a global system of connected networks.
- A **LAN** covers a small area, and a **WAN** covers a large area.
- Devices find each other with **IP addresses**. **DNS** looks up a **hostname** and gives back its IP address.
- Networks follow **standards**, such as **Ethernet** (wired) and **802.11** (wireless), set by organizations like the **IEEE**.
- **Architecture** is the network's design. In **client-server**, clients request resources from a central server. In **peer-to-peer (P2P)**, devices share resources directly.
- **Bandwidth** shows how much traffic a network can handle, measured in bits per second.
- Computers are networked for **resource sharing**, **flexibility**, **higher connectivity**, **data security and management**, **storage capacity**, and **entertainment**.

Stage 2: Network Fundamentals

LEARNING GOALS

By the end of this stage, you will be able to:

1. Describe how data travels between connected devices using networks.
2. Identify common network devices and explain what each one does.
3. Compare wired and wireless media.
4. Tell the difference between types of networks, such as LAN and WAN.
5. Describe star and mesh layouts (topologies).
6. Explain basic Internet ideas: TCP/IP, IP addresses, domain names, the Web, and URLs.

Topics covered:

- Devices (Computers, Server, Switch, Router, Modem, Access Point)
- Media (Wired, Wireless)
- Types of network (LAN, WAN)
- Layouts (Topologies: Star, Mesh)
- Internet fundamentals and other key Internet concepts

2.1 Devices

A computer network is built from parts that work together. The basic building blocks are:

Component	What it is
Node	Any device connected to the network, such as a computer, printer, or smartphone
Link	The **communication channel** that connects devices (cables, fiber optics, or wireless signals)
Protocol	The **set of rules** that decides how data is sent over the network
Network Interface Card (NIC)	A hardware part that **connects a computer to a network**. Every device that joins a network needs one (often built in).

Here are the main network devices:

Device	What it does	Easy way to think of it
Computer	An **end device** where people use the network (laptops, phones, desktops)	The people at the party
Server	A computer that **provides data or services** to other devices, such as web pages or files	The kitchen that serves everyone
Switch	**Connects devices inside one network** and sends data **only to the device it is meant for**	A mail clerk who delivers each letter to the right desk
Router	**Directs traffic between different networks**, such as your home LAN and the Internet	A road junction that sends traffic toward the right city
Modem	**Converts** digital data into a signal that can travel over a phone, cable, or fiber line, and **converts it back** at the other end	A translator between your network and your Internet line
Access Point (AP)	Lets **wireless devices** connect to a **wired network** using Wi-Fi	A doorway between cables and Wi-Fi

Some other devices you may meet:

Device	What it does
Hub	A basic device that **sends data to every connected device**, whether they need it or not (a switch is smarter because it sends data only to the right device)
Firewall	A security device that **monitors and filters** incoming and outgoing traffic using security rules
Gateway	An **entrance to another network**. It can **translate** between networks that use different protocols.
Repeater	**Boosts or regenerates a signal** so it can travel farther

(In many homes, one box combines a modem, a router, a switch, and an access point.)

[insert image about a network diagram with a modem, router, switch, access point, server, and several computers]

WORKED EXAMPLE

Task: Follow a laptop's request for a web page through a home network.

Step	Device	What happens
1	Laptop (NIC)	The laptop's NIC sends the request
2	Access point	The request travels over Wi-Fi to the access point, which passes it to the wired network
3	Switch	The switch sends it to the router, and only to the router
4	Router	The router sends it out toward the Internet
5	Modem	The modem converts the data into a signal for the Internet line
6	ISP and Internet	The request reaches the web server, which sends the page back along the same path

ACTIVITY

Name the device for each job:

a. Sends data only to the intended device inside one network. b. Directs traffic between your home network and the Internet. c. Lets a phone connect to a wired network using Wi-Fi. d. Provides web pages to other devices. e. Sends data to every device, whether needed or not.

Answer

a. **Switch** b. **Router** c. **Access point** d. **Server** e. **Hub**

2.2 Media

Network media (the plural of *medium*) are the **paths that carry data** between devices. These are the **links** of a network. There are two groups: **wired** and **wireless**.

Wired media use physical cables:

- **Ethernet cable (twisted-pair copper)**: The common cable for LANs. Data travels as **electrical signals**.
- **Fiber-optic cable**: Thin strands of glass. Data travels as **pulses of light**, which can carry a lot of data over long distances.

Wireless media use **radio waves** (or other signals) through the air instead of cables. The most common example is **Wi-Fi**.

Feature	Wired	Wireless
Speed	Usually fast and steady	Often slower, and can change
Reliability	High, less affected by interference	Can be affected by walls, distance, and other signals
Movement	Devices must stay near the cable	Devices can move freely
Setup	Needs cables	Easy, no cables
Security	Harder to tap into	Needs protection such as a password and encryption

Interference means other signals or objects that disturb a signal and make it weaker.

[insert image about an Ethernet cable, a fiber-optic cable, and a Wi-Fi signal]

WORKED EXAMPLE

Task: Choose the best medium.

Situation	Best medium	Why
A desktop computer in a gaming room that needs steady, fast speed	Ethernet cable	Fast and reliable
A phone and tablet moving around the house	Wi-Fi	Devices can move freely
A link carrying huge amounts of data between two buildings far apart	Fiber-optic cable	Light signals carry a lot of data over long distances

ACTIVITY

a. Which wired medium sends data as pulses of light? b. Give one advantage of wireless media and one of wired media. c. What is a link in a network?

Answer

a. **Fiber-optic cable** b. Wireless: devices can **move freely** and there are **no cables**. Wired: usually **faster and more reliable**. c. The **communication channel** that connects devices, such as a cable, a fiber, or a wireless signal.

2.3 Types of Network

Networks are grouped by **how large an area they cover** and **who can use them**.

Type	Full name	What it is	Example
LAN	Local Area Network	Covers a **small area**, such as an office or a home. Devices share resources like printers and files.	Your home Wi-Fi network
WAN	Wide Area Network	Covers a **large area**, such as cities or countries. The **Internet** is the largest WAN.	A company linking offices in different countries
MAN	Metropolitan Area Network	Spans **a city**. It is bigger than a LAN but smaller than a WAN.	A network connecting a city's libraries and offices
VPN	Virtual Private Network	Creates a **secure, encrypted connection** over a less secure network like the Internet. It is often used to reach a workplace's systems from far away.	An employee working from home who connects safely to the office network
Intranet		A **private network** that only an organization's **staff** can use, for internal communication and resources	A company's internal website for employees
Extranet		A controlled private network that gives **limited access to outsiders**, such as suppliers or partners, through a **secure login**	A supplier portal where partners track orders

(Encrypted means the data is scrambled so that only approved people can read it.)

[insert image about the sizes of networks: LAN, MAN, and WAN]

WORKED EXAMPLE

Task: Decide which network type fits each situation.

Situation	Type	Why
The computers in one school building	LAN	Small area
A university network joining campuses across a city	MAN	Spans a city
The global Internet	WAN	Covers the whole world
An employee safely reaching the company's files from a cafe	VPN	Secure connection over the Internet

ACTIVITY

a. What is the difference between a LAN and a WAN? b. Which network lets only a company's own staff use its internal website? c. Which network type gives suppliers limited, secure access?

Answer

a. A **LAN** covers a **small area** (like a home or office). A **WAN** covers a **large area** (like cities or countries). b. An **intranet** c. An **extranet**

2.4 Layouts (Topologies)

A **topology** is the **layout** of a network: how its devices are arranged and connected. Two common layouts are **star** and **mesh**.

STAR TOPOLOGY

In a **star**, every device connects to **one central device**, usually a **switch**. The shape looks like a star.

Advantages	Disadvantages
Easy to add or remove a device	If the **central device fails**, the **whole network stops**
If one device or cable fails, the **others keep working**	Needs a lot of cable to the center

MESH TOPOLOGY

In a **mesh**, devices are connected to **many other devices**. In a **full mesh**, **every device connects to every other device**.

Advantages	Disadvantages
Very **reliable**: if one link fails, data can take another path	**Expensive** and complicated, because it needs many links
Data has many possible routes	Harder to set up and manage

[insert image about a star topology and a mesh topology]

WORKED EXAMPLE

Task: Compare the number of links needed to connect devices.

- **Star** with *n* devices: each device has one link to the center, so **n links**.
- **Full mesh** with *n* devices: every pair of devices has a link, so **n × (n − 1) ÷ 2 links**.

Number of devices	Star links	Full mesh links
5	5	5 × 4 ÷ 2 = **10**
10	10	10 × 9 ÷ 2 = **45**

A full mesh needs many more links as the network grows, which is why it costs more.

ACTIVITY

a. In a star network, what happens if one computer's cable breaks? b. In a star network, what happens if the central switch fails? c. How many links does a full mesh of 6 devices need? d. Which layout is more reliable, star or mesh?

Answer

a. Only that computer loses its connection. The **others keep working**. b. The **whole network stops**. c. 6 × 5 ÷ 2 = **15 links** d. **Mesh**, because data can take another path if a link fails.

2.5 Internet Fundamentals and Other Key Concepts

TCP/IP AND IP ADDRESSES

TCP/IP (Transmission Control Protocol / Internet Protocol) is the **group of protocols used to connect devices on the Internet**. It controls how data is **broken into packets** (small pieces), **sent**, and **put back together**.

- **TCP**: Makes sure data arrives **reliably, in the right order, and without errors**.
- **IP**: Handles **addressing and routing** (choosing the path), so that each packet reaches the **correct destination**.

An **IP address** is a unique number for a device on a network. There are two versions:

Version	Address size	How many addresses
IPv4	**32 bits**	2^{32} = 4,294,967,296 (about 4.3 billion)
IPv6	**128 bits**	2^{128}, an enormously larger number

IPv6 was created because the world was running out of IPv4 addresses.

[insert image about data being split into packets by TCP and routed to a destination by IP]

WORKED EXAMPLE

Task: A file is sent from one computer to another.

Step	Protocol	What happens
1	TCP	Breaks the file into packets and numbers them
2	IP	Adds the destination IP address and sends each packet along a route
3	IP	The packets may take different routes across the network
4	TCP	At the destination, puts the packets back in order and asks for any missing ones to be sent again

ACTIVITY

a. Which protocol checks that data arrives in the correct order? b. How many bits is an IPv4 address? An IPv6 address?

Answer

a. **TCP** b. IPv4: **32 bits**. IPv6: **128 bits**.

DOMAIN NAMES AND DNS

A **domain name** is a **human-readable address** for a website, such as `www.example.com`. It is **mapped to the website's IP address** by the **Domain Name System (DNS)**, which **translates domain names into IP addresses** so that browsers can load Internet resources.
Parts of a domain name:

Part	Meaning	Example
Top-Level Domain (TLD)	The **last part** of the domain name	`.com`, `.org`, `.net`
Domain	The main name of the site	`example`
Subdomain	A part that goes **before** the domain, often used to organize sections of a website	`mail` in `mail.example.com`

WORKED EXAMPLE

Task: Break down `blog.example.org`.

Part	Value
Subdomain	`blog`
Domain	`example`
TLD	`.org`

ACTIVITY

Identify the TLD and the subdomain in `shop.mystore.com`.
Answer

TLD: **.com**. Subdomain: **shop**.

THE WORLD WIDE WEB, HTTP/HTTPS, AND URLS

The **World Wide Web** (the **Web**) is a system of **linked documents and media** that you reach over the Internet with a **web browser**. The Web is one service that runs on the Internet. The Internet is the network itself.

- **HTTP** (HyperText Transfer Protocol): The foundation of data exchange on the Web.
- **HTTPS** (HTTP Secure): The same, but with a layer of **encryption** added, so the communication is secure.
- **URL** (Uniform Resource Locator): The **unique address of a resource on the Web**. It tells the browser the **protocol**, the **domain**, and the **path** to the exact page.

WORKED EXAMPLE

Task: Break down the URL `https://www.example.com/news/today.html`.

Part	Value	Meaning
Protocol	`https`	Use secure HTTP
Domain	`www.example.com`	The website's name
Path	`/news/today.html`	The exact page on that site

ACTIVITY

a. What does the "S" in HTTPS add? b. Name the three parts of a URL described above.

Answer

a. **Encryption** (secure communication). b. The **protocol**, the **domain**, and the **path**.

OTHER KEY INTERNET CONCEPTS

Term	Meaning
Internet Service Provider (ISP)	A company that **provides access to the Internet**
Latency	The **delay** before data starts to transfer after you ask for it
FTP (File Transfer Protocol)	A protocol for **moving files between computers** on a network
SMTP	The protocol for **sending** email

Term	Meaning
POP3 (Post Office Protocol)	A protocol for **receiving** email by downloading it to your device
IMAP (Internet Message Access Protocol)	A protocol for **receiving** email while keeping it on the mail server

WORKED EXAMPLE

Task: Name the correct term.

Situation	Term
A pause between clicking a link and the page starting to load	Latency
The company whose service you pay for to get online	ISP
Uploading website files to a server	FTP

ACTIVITY

a. Which protocol is used to send email? b. What is latency?

Answer

a. **SMTP** b. The **delay** before data transfer begins after an instruction.

STAGE 2 SUMMARY

- **Nodes** are devices on the network. **Links** connect them, **protocols** are the rules, and a **NIC** connects a computer to the network.
- A **switch** connects devices inside one network, a **router** connects different networks, a **modem** converts signals for the Internet line, and an **access point** adds Wi-Fi to a wired network. A **server** provides services.
- **Wired** media (Ethernet, fiber-optic) are fast and reliable. **Wireless** media (Wi-Fi) let devices move freely.
- A **LAN** covers a small area, a **MAN** a city, and a **WAN** a large area. A **VPN** is a secure connection, an **intranet** is for staff only, and an **extranet** gives limited access to outsiders.
- A **star** topology uses one central device, and a **mesh** topology connects many devices to each other for greater reliability.

- **TCP/IP** breaks data into packets and delivers them. **DNS** turns domain names into IP addresses. The **Web** uses **HTTP/HTTPS** and **URLs**.

Chapter 4: Network Security

Stage 1: Security in Networks

LEARNING GOALS

By the end of this stage, you will be able to:

1. Describe how data travels between connected devices using networks.

Topics covered:

- Why it matters
- Privacy (confidentiality)
- Authentication
- Integrity
- Nonrepudiation
- Common protections (Firewall, Encryption)
- How data travels: the OSI model

OTHER KEY INTERNET CONCEPTS

Before we look at security, let's learn a few everyday terms about how the Internet works. A **network** is a group of connected devices that can share data, and the **Internet** is a huge network that connects networks all over the world.

Term	Meaning	Example
Internet Service Provider (ISP)	A company that gives you access to the Internet	The company you pay for home Wi-Fi
Latency	The delay before data starts to move after you ask for it	The pause between clicking a link and the page starting to load
FTP (File Transfer Protocol)	A set of rules for sending files from one computer to another over a network	Uploading a website's files to a server
SMTP (Simple Mail Transfer Protocol)	The rules used to **send** email	Pressing "Send" in your email app

Term	Meaning	Example
POP3 (Post Office Protocol)	Rules used to **receive** email by downloading it to your device	Your email app downloading new messages
IMAP (Internet Message Access Protocol)	Rules used to **receive** email while keeping it on the mail server, so you can read it from many devices	Reading the same inbox on your phone and laptop

A **protocol** is a set of agreed rules that devices follow so they can understand each other, just like two people agreeing to speak the same language.

[insert image about an ISP connecting a home to the Internet]

WORKED EXAMPLE

Task: You send an email from your laptop and later read the reply on your phone.

Step	What happens	Term used
1	Your laptop connects to the Internet through the company you pay for service	ISP
2	You press "Send" and the message is handed to the mail server	SMTP
3	There is a small wait before the message starts moving	Latency
4	Your phone reads the reply while the message stays on the server	IMAP

ACTIVITY

Name the term that matches each situation:

a. A video call freezes for a second before the other person's voice arrives. b. A company provides the Internet connection to your home. c. You want to read the same inbox on three different devices. d. You need to copy a large folder of files to a server.

Answer

a. **Latency** b. **ISP** c. **IMAP** d. **FTP**

1.1 Security

WHY IT MATTERS

When data travels across a network, other people or programs may try to look at it, change it, or pretend to be someone else. **Network security** means protecting data and devices from these dangers, both while the data is moving and while it is stored.

Good network security protects four things, and each one is covered in this stage:

Goal	Simple question it answers
Privacy	Can only the right people read my data?
Authentication	Is this person or device who they say they are?
Integrity	Has my data stayed the same?
Nonrepudiation	Can someone deny they sent it?

Think about online banking. You want your account details kept secret (privacy), the bank must know it's really you (authentication), the amount you transfer must not change on the way (integrity), and nobody should be able to say "I never sent that" (nonrepudiation).

[insert image about the four security goals around a data transfer: privacy, authentication, integrity, nonrepudiation]

WORKED EXAMPLE

Task: Match a security risk to the goal that stops it.

Risk	Goal that protects against it
A stranger reads your private messages	Privacy
Someone logs in to your account pretending to be you	Authentication
A hacker changes the amount in a payment	Integrity
A customer says "I never placed that order"	Nonrepudiation

ACTIVITY

Which security goal is in danger in each case?

a. Someone changes the grade in a file before it reaches the teacher. b. A stranger uses your friend's username to chat with you. c. Your password is read by a person sitting on the same public Wi-Fi.

Answer

a. **Integrity** (the data was changed) b. **Authentication** (the person is not who they claim to be)
c. **Privacy** (someone unauthorized can read your data)

PRIVACY (CONFIDENTIALITY)

Privacy (also called **confidentiality**) is the right of people to control who can see their personal information. In networks, this means keeping user data safe from unauthorized access while it is being sent and while it is stored. **Unauthorized** means not allowed or not given permission.

The main tool for privacy is **encryption**. Encryption is the process of scrambling data so that only approved people can read it. Data before encryption is called **plaintext**, and the scrambled version is called **ciphertext**. To read the ciphertext, you need a secret **key**, which is like a password that unlocks the message.

Two well-known encryption methods are:

- **AES (Advanced Encryption Standard)**: A fast method that uses the *same* key to lock and unlock data.
- **RSA**: A method that uses a *pair* of keys, one to lock the data and a different one to unlock it.

[insert image about plaintext being encrypted into ciphertext and decrypted with a key]

WORKED EXAMPLE

Task: Encrypt the word CAT by moving each letter forward by 1 place in the alphabet. (This simple trick is called a **Caesar cipher**. It is much weaker than AES or RSA, but it shows the idea.)

Letter	Move forward by 1	Result
C	→ D	D
A	→ B	B
T	→ U	U

Plaintext: CAT → Ciphertext: DBU
To decrypt it, the receiver, who knows the key ("move back by 1"), reverses the steps.

ACTIVITY

Using the same key (move forward by 1), encrypt the word `SUN`. Then decrypt `IBQQZ`.
Answer

- `SUN` → S→T, U→V, N→O → **TVO**
- `IBQQZ` → move each letter back by 1 → I→H, B→A, Q→P, Q→P, Z→Y → **HAPPY**

AUTHENTICATION

Authentication is the process of checking that a user or device really is who it claims to be. Common types are:

Type	How it works	Example
Username / Password	You enter something you know. This is the most common type.	Logging in to a website
Multi-Factor Authentication (MFA)	You must give more than one proof of identity.	A password plus a code sent to your phone
Biometric authentication	A unique body feature is used as proof.	Fingerprint or face recognition
Public Key Infrastructure (PKI)	A system that uses encryption and **digital certificates** (electronic ID cards for websites) to prove identity online.	The padlock icon in your browser

[insert image about the types of authentication: password, phone code, fingerprint, digital certificate]

WORKED EXAMPLE

Task: Log in to a school portal that uses MFA.

1. You type your username and password (**something you know**).
2. The portal sends a 6-digit code to your phone.
3. You enter the code (**something you have**).
4. The portal lets you in only if both steps are correct.

Even if someone steals your password, they still cannot log in without your phone.

ACTIVITY

Name the type of authentication in each case:

a. Unlocking a phone with your fingerprint. b. Typing a password and then entering a code sent by text message. c. A browser checking a website's digital certificate before connecting.

Answer

a. **Biometric authentication** b. **Multi-Factor Authentication (MFA)** c. **Public Key Infrastructure (PKI)**

INTEGRITY

Integrity means making sure data stays exactly the same while it is sent or stored, with no accidental or deliberate changes. Two common tools are:

- **Hash function**: A math recipe that turns data of any size into a short, fixed-size value called a **hash** (or **digest**). The same data always gives the same hash. If even one character changes, the hash changes too, which shows the data was tampered with. **SHA-256** is a popular hash function.
- **Checksum**: A simpler check. A number is worked out from the data before sending, then worked out again after receiving. If the two numbers differ, something went wrong.

[insert image about a hash function turning data into a fixed-size hash value]

WORKED EXAMPLE

Task: Use a simple checksum (add up the digits) to check that the data `4 7 2 9` arrived safely.

	Data	Checksum (sum of digits)
Sent	4 7 2 9	4 + 7 + 2 + 9 = **22**
Received	4 7 2 9	4 + 7 + 2 + 9 = **22**

The checksums match, so the data was not changed.

Now imagine it arrives as `4 7 3 9`: the checksum is 4 + 7 + 3 + 9 = **23**. This does not match 22, so we know the data was changed.

ACTIVITY

The data `1 5 8 3` is sent with a checksum of 17. The receiver gets `1 5 8 4`.
a. What checksum does the receiver calculate? b. Was the data changed?

Answer

a. 1 + 5 + 8 + 4 = **18** b. **Yes.** 18 does not match 17, so the data was changed on the way.

NONREPUDIATION

Nonrepudiation means that a person cannot deny sending a message or signing something. (To **repudiate** means to deny.) It gives proof of who did what. Two common tools are:

- **Digital signatures**: A signature made with encryption that proves **who sent** a message and that the message was **not changed**.
- **Log files**: Records that automatically list events, such as who logged in and when. They can be used as evidence that an action took place.

[insert image about a digital signature attached to a message]

WORKED EXAMPLE

Task: A customer places an online order and later claims they never did.

Evidence	What it shows
Digital signature on the order	The order was created with the customer's private key, so it came from them
Log file entry: "Order #482, 10:32 AM, Account: sam123"	The order was made from Sam's account at that time

Together, this evidence makes it hard for the customer to deny placing the order.

ACTIVITY

A manager emails a signed approval for a purchase and later says, "I never approved that." Which two tools could prove the approval happened, and what does each one show?

Answer

- **Digital signature**: proves the approval came from the manager and was not changed.
- **Log file**: records when the email was sent and from which account.

COMMON PROTECTIONS (FIREWALL, ENCRYPTION)

Besides the tools above, networks use some everyday protections:

- **Firewall**: A security system that watches the data going in and out of a network and **blocks anything that breaks its rules**. You can think of it as a security guard at the entrance of a building. A firewall can be a program or a physical device.
- **Encryption**: Scrambling data so only approved people can read it (see *Privacy* above). When you visit a website whose address starts with `https`, your data is encrypted on its way.

[insert image about a firewall sitting between the Internet and a private network]

WORKED EXAMPLE

Task: A school network has a firewall with this rule: "*Block all traffic from unknown websites.*"

Incoming data	Firewall decision	Reason
A student opens a known learning website	Allowed	The site is trusted
A message arrives from an unknown website	Blocked	It matches the "unknown website" rule

ACTIVITY

Decide whether a firewall or encryption is the better protection:

a. You want to stop unknown programs from entering your home network. b. You want a stranger on public Wi-Fi to be unable to read your messages.

Answer

a. **Firewall**: it filters what enters or leaves the network. b. **Encryption**: even if the data is intercepted, it can't be read without the key.

1.2 OSI Model

To describe how data travels from one device to another, we use the **OSI model**. OSI stands for **Open Systems Interconnection**. It is a **conceptual model** (an idea used to explain how something works, not a physical machine) created by the **International Organization for Standardization (ISO)**.

In plain words, the OSI model is a standard that lets different computer systems talk to each other. You can think of it as a universal language for networking. It splits communication into **seven layers**, stacked one on top of the other. Each layer has its own job and passes the data to the next one.

Layer	Name	Main job
7	**Application**	Where software such as browsers and email apps reaches the network services

Layer	Name	Main job
6	**Presentation**	Makes sure data is in a usable form; handles encryption
5	**Session**	Opens, keeps, and closes connections between devices
4	**Transport**	Sends data using protocols such as TCP and UDP
3	**Network**	Chooses the path the data will take
2	**Data Link**	Sets the format of data on the network
1	**Physical**	Sends the raw bit stream over cables or other physical media

[insert image about the OSI model diagram with its seven layers]

LAYER 1: THE PHYSICAL LAYER

This layer is the **physical hardware** used to send data, such as cables and switches. A **bit** is a single `0` or `1`, and a **bit stream** is a long line of bits. This layer turns data into a bit stream and sends it as signals. Both devices must agree on a **signal standard** so they can tell the 1s from the 0s.
[insert image about a sending cable transmitting a bit stream to a receiving cable]

LAYER 2: THE DATA LINK LAYER

This layer moves data between devices **inside the same network**. It takes **packets** (small chunks of data) from the network layer and splits them into even smaller pieces called **frames**. It also handles **flow control** (not sending data faster than the receiver can take it) and **error control** (spotting and fixing mistakes) within that network.

[insert image about frame creation and transfer between devices on the same network]

LAYER 3: THE NETWORK LAYER

This layer moves data **between different networks**. It is not needed if both devices are on the same network. On the sender's side, it breaks the data it receives into **packets**, and on the receiver's side, it puts them back together. It also finds the best path for the data to follow, which is called **routing**. Protocols used here include **IP**, ICMP, IGMP, and IPsec.

[insert image about packet creation, transport, and reassembly]

LAYER 4: THE TRANSPORT LAYER

This layer handles **end-to-end communication**, meaning from the sending device all the way to the receiving device. It breaks data from the session layer into **segments** and passes them down. On the receiving end, it puts the segments back together. It also controls the speed of sending (flow control) and checks that all data arrived, asking for it to be sent

again if something is missing (error control). Protocols here include **TCP** (careful and reliable) and **UDP** (fast but without double-checking).

[insert image about segmentation, transport, and reassembly]

LAYER 5: THE SESSION LAYER

A **session** is the period when two devices are connected and talking. This layer opens the session, keeps it open long enough to send all the data, and closes it afterward so resources aren't wasted. It also sets **checkpoints**, which are like save points. If the connection crashes, the transfer can restart from the last checkpoint instead of from the beginning.

[insert image about a communication session between two devices]

LAYER 6: THE PRESENTATION LAYER

This layer prepares data for the application layer. It handles three jobs:

- **Translation**: converting data between different formats so the receiving device can understand it.
- **Encryption**: scrambling data at the sender's end and unscrambling it at the receiver's end.
- **Compression**: making data smaller so it travels faster.

[insert image about encryption, compression, and translation]

LAYER 7: THE APPLICATION LAYER

This layer works directly with the user's data. It lets software such as web browsers and email apps start communication. The software itself is **not** part of this layer; the layer manages the protocols those programs use. Key protocols are **HTTP** (for web pages) and **SMTP** (for sending email).

[insert image about a browser requesting content from a website through the application layer]

WORKED EXAMPLE

Task: Follow what happens when you open a web page and your request is sent out.

Step	Layer	What happens
1	7 – Application	Your browser asks for a web page using HTTP
2	6 – Presentation	The request is encrypted and compressed
3	5 – Session	A session is opened between your device and the website

Step	Layer	What happens
4	4 – Transport	The data is split into segments (TCP)
5	3 – Network	Segments become packets, and a path is chosen
6	2 – Data Link	Packets are turned into frames
7	1 – Physical	The frames are sent as a bit stream through the cable or Wi-Fi

The receiving device does the same steps **in reverse order** (layer 1 up to layer 7) to rebuild the original request.

ACTIVITY

Name the OSI layer for each job:

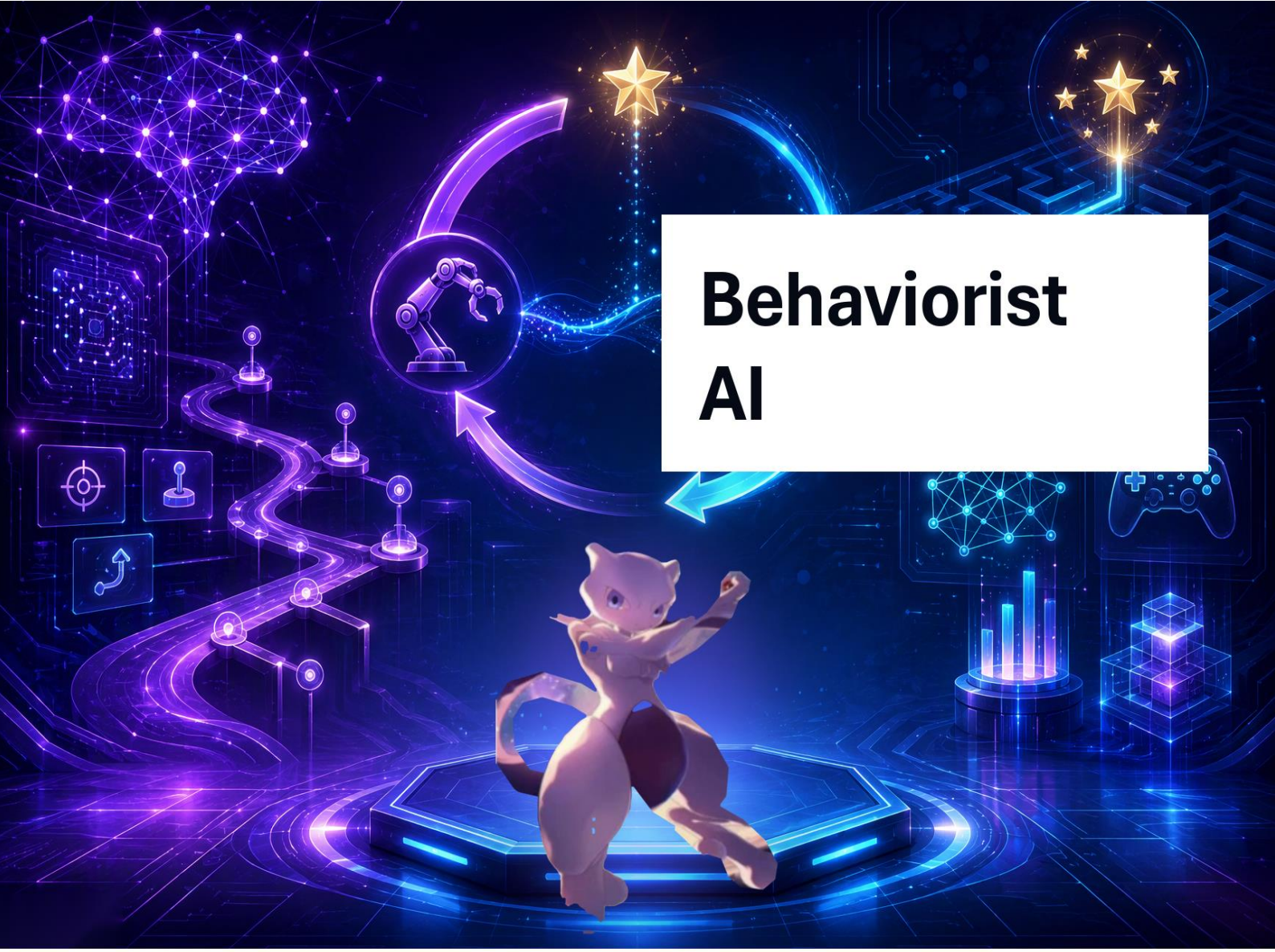
a. Choosing the best path for the data to reach another network. b. Turning data into a bit stream sent through a cable. c. Compressing and encrypting data. d. Saving checkpoints so a crashed download can continue. e. Splitting data into segments and checking that all of it arrived.

Answer

a. **Layer 3: Network layer** (routing) b. **Layer 1: Physical layer** c. **Layer 6: Presentation layer** d. **Layer 5: Session layer** e. **Layer 4: Transport layer**

STAGE 1 SUMMARY

- An **ISP** gives you Internet access, **latency** is the delay before data starts moving, and **protocols** such as FTP, SMTP, POP3, and IMAP are agreed rules for transferring files and email.
- **Network security** protects data while it travels and while it is stored.
- **Privacy** keeps data secret using **encryption** (for example AES and RSA).
- **Authentication** checks identity using passwords, MFA, biometrics, or PKI.
- **Integrity** makes sure data is unchanged, using **hash functions** and **checksums**.
- **Nonrepudiation** stops people from denying their actions, using **digital signatures** and **log files**.
- A **firewall** blocks unwanted traffic, and **encryption** hides data from outsiders.
- The **OSI model** explains how data travels using seven layers: Physical, Data Link, Network, Transport, Session, Presentation, and Application.



Behaviorist AI

Learning Goals

By the end of this module, you will be able to:

- 1) Define Behaviorist AI and describe its basic process with an example.
- 2) Explain the core loop of Reinforcement Learning (agent, environment, actions, and rewards).
- 3) Distinguish between model-based and model-free learning.
- 4) Describe the parts of a Markov Decision Process (MDP): states, actions, rewards, and policy.
- 5) Explain Q-learning conceptually and understand the balance between exploration and exploitation.
- 6) Describe how Reinforcement Learning is used in robotics and games, and give examples of real-world applications.
- 7) Identify the main challenges (like reward design) and limitations (like safety and explainability) of Behaviorist AI.

Chapter 5: Intro to Behaviorist AI & RL

OVERVIEW & PREDICTION

Imagine you are teaching a dog a new trick, like rolling over. You do not hand the dog a 300-page manual on canine anatomy, nor do you write out code for its brain to execute. Instead, you wait for the dog to make a move. When it gets closer to what you want, you give it a treat. When it barks or runs away, you withhold the treat. Over time, without reading a single word, the dog learns a complex sequence of physical movements purely through trial, error, and incentives.

Predict: If an artificial intelligence is placed in a digital maze without a map, rules, or instructions, how might it figure out the shortest path to the exit using only a score counter that goes up when it reaches the exit and down every time it hits a wall?

LEARNING GOALS

By the end of this chapter, you will be able to:

- Define behaviorist AI and explain its connection to psychology and neuroscience.
- Diagram the core interaction loop between an agent and its environment.
- Identify the four key components of Reinforcement Learning (Agent, Environment, Actions, and Rewards).
- Contrast Model-Based learning with Model-Free learning using real-world analogies.

---

1. What is Behaviorist AI?

Behaviorist AI is a branch of artificial intelligence inspired by psychological behaviorism, where an artificial system learns how to act by interacting with its surroundings and receiving feedback rather than following pre-programmed rules.

In classical psychology, behaviorism (popularized by figures like B.F. Skinner) suggests that actions followed by positive outcomes tend to be repeated, while actions followed by negative outcomes tend to decline. Neuroscience shows a similar pattern in living brains, where neurotransmitters like dopamine reinforce neural pathways associated with successful choices.

Behaviorist AI translates this biological concept into mathematics. Instead of giving a machine step-by-step instructions (Symbolic AI) or showing it millions of labeled photos (Supervised Learning), we place the machine in an environment and let it learn through trial and error.

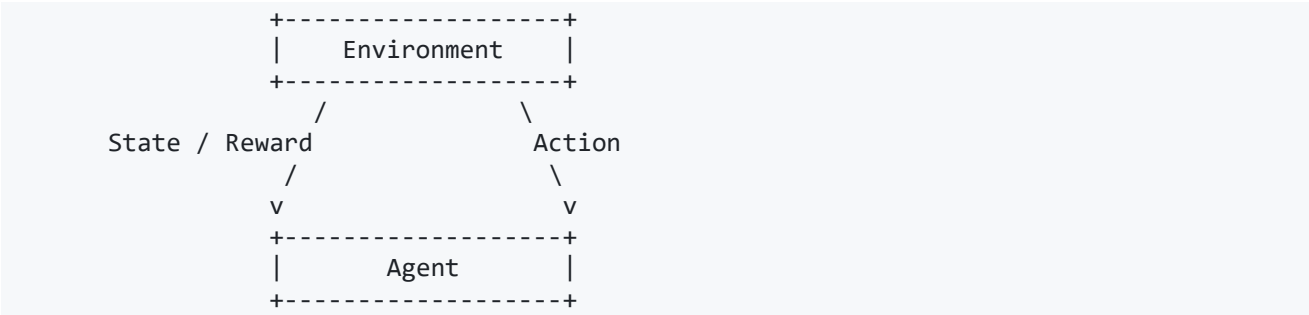
Check Your Understanding: In your own words, explain how Behaviorist AI differs from traditional software programming where a developer writes strict "if/then" rules.

---

2. The Core Process: Reinforcement Learning

The primary framework used to implement Behaviorist AI is **Reinforcement Learning**, a machine learning paradigm where a computer program learns to make decisions by performing actions and receiving rewards or penalties.

To understand Reinforcement Learning, we divide any system into four primary components:



1. **Agent**: The autonomous decision-maker or AI learner that perceives the environment and takes actions.
2. **Environment**: Everything outside the agent with which the agent interacts, responds to actions, and presents new situations.
3. **Action**: The set of all possible moves, decisions, or behaviors that the agent can execute at any given moment.
4. **Reward**: A numeric scalar feedback signal sent from the environment to the agent indicating how successful or desirable the previous action was.

The agent's goal is not just to get an immediate reward, but to maximize its total cumulative reward over time.

Check Your Understanding: Imagine an AI learning to play chess. Identify the **Agent**, **Environment**, **Action**, and **Reward** in this specific context.

---

3. Model-Based vs. Model-Free Learning

When an agent learns within an environment, it generally takes one of two strategic approaches:

Model-Based Learning is a reinforcement learning approach where the agent builds an internal mental representation or map of how the environment works to predict future outcomes before taking an action. Think of this like playing a game of chess while carefully planning five moves ahead in your head, predicting how your opponent will react to each choice.

Model-Free Learning is a reinforcement learning approach where the agent learns directly from trial-and-error experience without attempting to build an explicit model or map of the environment. Think of this like learning to ride a bicycle. You do not calculate physics equations or model gravity in your head; you simply adjust your balance based on immediate feedback until muscle memory takes over.

Feature	Model-Based	Model-Free
Internal Map	Builds a detailed map/rules of the environment	Operates purely on experience
Planning Ability	High (can simulate future states)	Low (reacts based on past learned values)
Computational Needs	High memory and processing for planning	Faster execution once trained

Check Your Understanding: Predict which approach (Model-Based or Model-Free) a fast-moving reflex system in a self-driving car might rely on when deciding to slam on the brakes for an unexpected obstacle.

HANDS-ON ACTIVITY (APPLY)

GOAL

Design a complete Reinforcement Learning feedback framework for a real-world task on paper.

PREPARATION

Grab a sheet of paper and a pen (or open a blank text document).

TASK

Select **one** of the following scenarios:

1. An AI system learning to optimize household thermostat temperatures to lower electricity bills.
2. An AI vacuum cleaner learning to clean a room as quickly as possible.
3. An AI character in a video game learning to dodge incoming projectiles.

For your chosen scenario, write out explicit definitions for each of the following:

- **The Agent**
- **The Environment**
- **At least 3 distinct Actions**
- **A specific Reward (+ points) condition**
- **A specific Penalty (- points) condition**

OBSERVATION

Look closely at your reward and penalty conditions. Ask yourself: Is there any way the AI could "cheat" your reward system to get points without actually completing the real-world goal? (For example, could the vacuum just run around in circles over a clean spot to gain movement points?)

REFLECTION

How might changing the penalty value (making it much harsher or much milder) affect how cautiously or aggressively your AI agent behaves?

FEEDBACK

Check your setup against these standard guidelines:

- The **Agent** must be the software making choices, not the human user.
- The **Environment** must include the physical or virtual space the agent interacts with.
- The **Reward/Penalty** must directly encourage the desired outcome while preventing unintended loopholes.

Summary & Glossary

Behaviorist AI turns the challenge of programming complex behaviors into a process of trial, error, and feedback. Rooted in psychological conditioning and neuroscience, this branch of AI relies heavily on Reinforcement Learning. An autonomous agent constantly observes its environment, takes actions, and receives numerical rewards or penalties. Depending on whether the agent constructs an internal map of its surroundings, it utilizes either Model-

Based or Model-Free learning strategies to continuously improve its decision-making capabilities.

GLOSSARY

- **Behaviorist AI**: A branch of artificial intelligence inspired by psychological behaviorism, where an artificial system learns how to act by interacting with its surroundings and receiving feedback rather than following pre-programmed rules.
- **Reinforcement Learning**: A machine learning paradigm where a computer program learns to make decisions by performing actions and receiving rewards or penalties.
- **Agent**: The autonomous decision-maker or AI learner that perceives the environment and takes actions.
- **Environment**: Everything outside the agent with which the agent interacts, responds to actions, and presents new situations.
- **Action**: The set of all possible moves, decisions, or behaviors that the agent can execute at any given moment.
- **Reward**: A numeric scalar feedback signal sent from the environment to the agent indicating how successful or desirable the previous action was.
- **Model-Based Learning**: A reinforcement learning approach where the agent builds an internal mental representation or map of how the environment works to predict future outcomes before taking an action.
- **Model-Free Learning**: A reinforcement learning approach where the agent learns directly from trial-and-error experience without attempting to build an explicit model or map of the environment.

Chapter 6: RL Algorithms and MDPs

LEARNING GOALS

By the end of these stages, you will be able to:

1. Explain basic principles of how AI models use data to generate predictions.

Stage	Topics
Stage 1: Markov Decision Processes (MDPs)	• Markov Decision Process (1): States, Actions, Rewards • Markov Decision Process (2): Policy
Stage 2: RL Algorithms and Techniques	• Reinforcement Learning Algorithms (overview) • Value-Based (Q-Learning conceptual overview) • Exploration vs. Exploitation

---

Background: AI and Machine Learning

Artificial Intelligence (AI) is the part of computer science that builds machines and programs able to do tasks that normally need human intelligence, such as understanding language, making decisions, solving problems, and recognizing things they see.

Machine Learning (ML) is a part of AI where computers **learn from data** and make predictions, instead of being told exactly what to do for every task. Three key ideas appear in every ML system:

- **Training data**: The examples the computer learns from.
- **Model**: What the computer learns. It is a summary of the patterns found in the data, and it uses those patterns to make predictions.
- **Algorithm**: The step-by-step method the computer follows to learn. It adjusts the model again and again so the predictions get better.

There are three main types of machine learning:

Type	How it learns	Example
Supervised learning	Learns from **labeled data**, which means examples that already come with the right answer (the **label**). It learns to match inputs to outputs.	Photos labeled "cat" or "dog" teach the model to name new photos
Unsupervised learning	Learns from **unlabeled data**, with no right answers given. It finds patterns or groups (called **clusters**) on its own.	Grouping customers who buy similar products
Reinforcement learning (RL)	Learns by **trial and error**, getting rewards for good choices and penalties for bad ones.	A robot learning to walk

[insert image about supervised learning: labeled shapes train a model, which then predicts test shapes]

[insert image about unsupervised learning: unlabeled data goes into a machine and produces groups]

The main differences between supervised and unsupervised learning:

- Supervised learning **needs labeled data**; unsupervised learning does not.
- Supervised learning is used to **predict an answer**; unsupervised learning is used to **explore and understand** the data.
- Supervised learning uses methods like linear regression and neural networks; unsupervised learning uses methods like k-means clustering.

[insert image about a graph showing supervised learning (line of best fit) next to unsupervised learning (clusters)]

Almost every ML project follows the same **workflow**:

1. **Data collection**: Gather the data needed.
2. **Data preprocessing**: Clean and prepare the data.
3. **Model selection and training**: Pick an algorithm and train the model.
4. **Model evaluation**: Test the model on **unseen data** (data it has not learned from) to check it works well.
5. **Model deployment**: Use the trained model in the real world.

WORKED EXAMPLE

Task: Decide which type of learning fits each problem.

Problem	Type	Why
Predict whether an email is spam, using thousands of emails already marked "spam" or "not spam"	Supervised	The data has labels
Sort news articles into groups of similar topics, with no topics given	Unsupervised	No labels, the model finds groups
Teach a program to play a video game by giving points for winning	Reinforcement	Learns from rewards and penalties

ACTIVITY

Name the type of learning (supervised, unsupervised, or reinforcement) for each case:

a. A model learns to read handwritten numbers from images that each have the correct number attached. b. A store wants to find groups of similar shoppers, but has no categories ready. c. A robot vacuum gets a reward each time it cleans a new area without bumping into furniture.

Answer

a. **Supervised** (labeled examples) b. **Unsupervised** (no labels, finding groups)
c. **Reinforcement** (learning from rewards and penalties)

Stage 1: Markov Decision Processes (MDPs)

REINFORCEMENT LEARNING IN BRIEF

In reinforcement learning, a program learns by **doing**. It tries something, sees what happens, and gets feedback. The main parts are:

Part	Meaning	Example (robot learning to walk)
Agent	The decision-maker that learns	The robot
Environment	The world the agent interacts with	The floor and everything around the robot
State	The current situation of the environment	The robot's leg positions and balance

Part	Meaning	Example (robot learning to walk)
Action	A choice the agent makes in a state	Move the left leg forward
Reward	Feedback for an action (positive is good, negative is a penalty)	+1 for a step, −5 for a fall

The agent repeats a loop: it looks at the **state**, picks an **action**, the environment responds, and the agent receives a **reward** and a new state.

[insert image about the reinforcement learning loop: Agent -> Action -> Environment -> Reward and new State -> Agent]

Reinforcement learning has been used in many fields:

- **Games**: Google's **AlphaGo** beat top Go player Lee Sedol in 2016. IBM's **Deep Blue** also beat world chess champion Garry Kasparov in 1997, although Deep Blue relied mostly on searching through moves rather than on learning from rewards.
- **Robotics**: Robots learn to walk by trying movements and being rewarded for steps.
- **Self-driving cars**: Cars learn to drive safely, with rewards for safe driving and penalties for accidents.
- **Finance**: Trading programs learn to make better trades from market data.

[insert image about Deep Blue vs. Garry Kasparov and AlphaGo vs. Lee Sedol]

WORKED EXAMPLE

Task: Identify the parts of reinforcement learning for a self-driving car.

Part	In this example
Agent	The car's driving program
Environment	Roads, traffic, and pedestrians
State	Current speed, lane, and distance to other cars
Action	Speed up, slow down, or turn
Reward	Positive for safe driving, negative for accidents or breaking traffic rules

ACTIVITY

A program learns to play a simple game where it moves a character to collect coins and avoid traps. Name the agent, environment, one possible action, and one possible reward.

Answer

- **Agent**: the program controlling the character
- **Environment**: the game world with coins and traps
- **Action**: move left, right, up, or down (or jump)
- **Reward**: positive for collecting a coin, negative for hitting a trap

---

MARKOV DECISION PROCESS (1): STATES, ACTIONS, REWARDS

A **Markov Decision Process (MDP)** is a mathematical way of describing a decision problem where results are **partly random and partly controlled by the decision-maker**. Reinforcement learning problems are usually written as MDPs.

An MDP has these building blocks:

Component	Symbol	Meaning
States	S	All the different situations that can happen
Actions	A	The choices available in each state
Transition model	P	The **probability** (chance) of ending up in each new state after taking an action
Reward function	R	The immediate reward received after an action moves the agent from one state to another
Policy	π	The strategy for which action to pick in each state (covered in the next section)

[insert image about the Markov Decision Process components: states, actions, transition model, rewards, policy]

The word **Markov** means that the next state depends only on the **current state** and the action taken, not on everything that happened before.

WORKED EXAMPLE

Task: Describe a traffic light as an MDP.

Component	Traffic light example
States (S)	Red, Yellow, Green
Actions (A)	Stay in the current color, or switch to the next color
Transition model (P)	Set by rules such as how long each light lasts. For example, if the light is Green and the action is "switch", it moves to Yellow with a probability of 100%
Reward function (R)	Positive reward when traffic flows smoothly, negative reward when cars are stuck in a long queue

The goal is to pick actions that keep traffic flowing smoothly.

ACTIVITY

A robot moves along a hallway with three spots: **A**, **B**, and **C**. It can **move right** or **stay**. Reaching spot C gives a reward of +10, and every other step gives −1.

a. List the states. b. List the actions. c. What is the reward for moving from A to B? d. What is the reward for moving from B to C?

Answer

a. States: **A, B, C** b. Actions: **move right, stay** c. **−1** (any step that does not reach C)
d. **+10** (reaching spot C)

MARKOV DECISION PROCESS (2): POLICY

A **policy** is the agent's strategy: a rule that says **which action to take in each state**. It is written as π(S) → a, which reads "in state S, take action a". The best possible policy, which collects the most reward over time, is called the **optimal policy** and is written π*.

To find a good policy, the agent needs to know how good each state is. A **value function** estimates the **total reward the agent can expect to collect in the future** starting from a given state. States that lead to big rewards get high values.

Future rewards are usually counted a little less than immediate ones, because rewards now are more certain. The number used for this is called the **discount factor** (written γ, a value between 0 and 1). The total reward, called the **return**, is:

Return = $R_1 + \gamma \cdot R_2 + \gamma^2 \cdot R_3 + \ldots$

There are two main methods for solving an MDP:

- **Value iteration**: Update the value of every state again and again until the values stop changing (this is called **convergence**). Then pick the best action for each state.
- **Policy iteration**: Switch between two steps: **evaluate** how good the current policy is, then **improve** it. Repeat until the policy can't be improved.

[insert image about a policy: arrows in each state showing the chosen action]

WORKED EXAMPLE

Task: Work out the return for rewards of 2, 4, and 10 using a discount factor of 0.9.

Step	Reward	Discount	Value counted
1	2	1	2
2	4	0.9	3.6
3	10	0.9 × 0.9 = 0.81	8.1

Return = 2 + 3.6 + 8.1 = **13.7**

Policy example (traffic light):

State	Policy (action to take)
Green, and the queue on the other road is long	Switch to Yellow
Green, and few cars are waiting	Stay Green
Red, and the time is up	Switch to Green

ACTIVITY

a. A robot gets rewards of 1, 2, and 3 over three steps. With a discount factor of 0.5, what is the return? b. Write a simple policy for a robot vacuum with two states: **Battery low** and **Battery OK**.

Answer

a. 1 + (0.5 × 2) + (0.25 × 3) = 1 + 1 + 0.75 = **2.75**

b. One possible policy:

State	Action
Battery low	Go back to the charging station
Battery OK	Keep cleaning

Any policy that gives one clear action for each state is correct.

Stage 2: RL Algorithms and Techniques

REINFORCEMENT LEARNING ALGORITHMS (OVERVIEW)

In a real problem, the agent usually does **not** know the transition model or rewards ahead of time. It must learn them by playing. A **learning algorithm** is the method the agent uses to update its **policy** and its **value function** based on the rewards it receives.

RL algorithms are often grouped by what they learn:

Group	What the agent learns	Idea
Value-based	How valuable each state or action is	Pick the action with the highest value
Policy-based	The policy directly	Learn which action to take without first scoring everything
Model-based	A model of how the environment works	Plan ahead using the model

In this stage, we focus on the **value-based** group.

[insert image about three groups of RL algorithms: value-based, policy-based, model-based]

WORKED EXAMPLE

Task: Match each description to a group.

Description	Group
"I keep a score for every action and choose the highest one."	Value-based

Description	Group
"I learn directly which action to pick in each situation."	Policy-based
"I build a map of how the world reacts, then plan with it."	Model-based

ACTIVITY

Which group does each agent belong to?

a. An agent that writes down a score for every move in a board game and picks the best score. b. An agent that practices a robot arm movement and directly adjusts its action rules.

Answer

a. **Value-based** b. **Policy-based**

VALUE-BASED LEARNING: Q-LEARNING (CONCEPTUAL OVERVIEW)

Q-learning is a popular value-based algorithm. The "Q" stands for **quality**, meaning how good an action is. The agent keeps a table called a **Q-table**, which stores a **Q-value** for every pair of *(state, action)*. A higher Q-value means a better choice.

At first, all Q-values are zero. As the agent plays, it updates the Q-value of the action it just used:

New Q = Old Q + α × (Reward + γ × Best next Q − Old Q)

Here is what each part means:

- **α (alpha)**, the **learning rate**: How much the agent changes its mind after each experience (between 0 and 1).
- **γ (gamma)**, the **discount factor**: How much future rewards matter.
- **Best next Q**: The highest Q-value available in the next state.

In simple words: the agent compares **what it expected** with **what it actually got** and moves its estimate a little closer to the real result.

State	Action: Left	Action: Right
A	0	0

State	Action: Left	Action: Right
B	0	0
C	0	0

(An empty Q-table at the start of learning.)

[insert image about a Q-table being updated as the agent moves between states]

WORKED EXAMPLE

Task: The agent is in state B and takes action "Right". It receives a reward of 10. The best Q-value in the next state is 0. Use $\alpha = 0.5$, $\gamma = 0.9$, and an old Q-value of 0.

1. Calculate the target: Reward + γ × Best next Q = 10 + (0.9 × 0) = **10**
2. Find the difference from the old value: 10 − 0 = **10**
3. Multiply by the learning rate: 0.5 × 10 = **5**
4. New Q = 0 + 5 = **5**

The Q-value for (B, Right) is now **5**, so the agent sees this action as worth taking.

ACTIVITY

The old Q-value for (state C, "Left") is 5. The agent receives a reward of 2, and the best Q-value in the next state is 10. Use $\alpha = 0.5$ and $\gamma = 0.9$. What is the new Q-value?

Answer

1. Target = $2 + (0.9 \times 10) = 2 + 9 =$ **11**
2. Difference = $11 - 5 =$ **6**
3. Multiply by α: $0.5 \times 6 =$ **3**
4. New Q = $5 + 3 =$ **8**

EXPLORATION VS. EXPLOITATION

An agent faces a tricky choice every time it acts:

- **Exploration**: Try a **new** action to discover whether it might give a better reward.
- **Exploitation**: Choose the action that is **already known** to give a good reward.

If the agent only exploits, it may miss better options. If it only explores, it never takes advantage of what it has learned. A good agent balances the two.

A simple way to balance them is the **ε-greedy** method (ε is the Greek letter *epsilon*). The agent picks a small number ε, such as 0.1:

- With probability **ε** (10% of the time), it **explores** by picking a random action.
- With probability **1 − ε** (90% of the time), it **exploits** by picking the action with the highest Q-value.

Often, ε starts high and becomes smaller as the agent learns more.

[insert image about exploration vs. exploitation: trying a new path versus taking the known best path]

WORKED EXAMPLE

Task: You are choosing where to eat lunch. Your favorite café gives a reward of 8 out of 10 every time.

Choice	Type	Result
Go to your favorite café	Exploitation	A reliable 8
Try a new restaurant	Exploration	Could be a 4, or it could be a 10

If you always go to the café, you may never find a restaurant you like even more. If you always try something new, you often get meals worse than your favorite. With ε = 0.1, you would try something new about **1 day out of every 10**.

ACTIVITY

An agent uses ε-greedy with ε = 0.2.

a. What percentage of the time does it explore? b. What percentage of the time does it exploit? c. Is "choosing the action with the highest Q-value" exploration or exploitation?

Answer

a. **20%** b. **80%** $(1 - 0.2)$ c. **Exploitation**

RELATED IDEAS: DEEP LEARNING, NEURAL NETWORKS, AND NLP

Some AI systems learn in a much deeper way:

- **Neural network**: A model built to loosely copy the human brain. It is made of connected **nodes** (also called **neurons**) that pass information along, process it, and produce a prediction.

- **Deep learning**: A type of machine learning that uses neural networks with **many layers**. The layers between the input and the output are called **hidden layers**. More layers let the model learn more complex patterns.

[insert image about a neural network with an input layer, hidden layers, and an output layer]

When reinforcement learning is combined with deep learning, an agent can learn from very complex situations, such as the images on a video game screen.

Natural Language Processing (NLP) is the part of AI that helps machines **understand, interpret, and create human language**. It is used on **unstructured data** such as reviews, call transcripts, and social media posts (data that does not fit neatly into rows and columns). NLP has three key parts:

Part	Meaning	Example
Computational linguistics	Uses language science and computing to study human language with computers	Studying how sentences are built
Natural Language Understanding (NLU)	Machines figure out meaning and context from text and speech	A voice assistant understanding your question
Natural Language Generation (NLG)	Machines write or speak human-like language	A program writing a short report or speaking an answer

WORKED EXAMPLE

Task: A customer writes the review: "The battery dies too fast." A company uses NLP to handle it.

Step	NLP part	What happens
1	NLU	The system works out that the customer is unhappy about **battery life**
2	NLG	The system writes a reply apologizing and offering help

ACTIVITY

Decide whether each task is NLU or NLG:

a. A smart speaker figures out that you asked for tomorrow's weather. b. A program turns sales numbers into a written summary.

Answer

a. **NLU** (understanding speech) b. **NLG** (creating text)

SUMMARY

- **Machine learning** lets computers learn from data using **training data**, **models**, and **algorithms**. The three types are **supervised**, **unsupervised**, and **reinforcement** learning.
- In **reinforcement learning**, an **agent** takes **actions** in an **environment**, moves between **states**, and receives **rewards**.
- A **Markov Decision Process (MDP)** describes the problem with **states, actions, a transition model, and a reward function**.
- A **policy** says which action to take in each state. A **value function** estimates future reward, and the **discount factor** makes later rewards count a little less.
- **Value iteration** and **policy iteration** are two ways to solve an MDP.
- **Q-learning** keeps a **Q-table** of scores for each (state, action) pair and updates it after every experience.
- Agents must balance **exploration** (trying new things) with **exploitation** (using what they know), for example with the **ε-greedy** method.
- **Neural networks** and **deep learning** let models learn complex patterns, and **NLP** helps machines work with human language.

Chapter 7: AI in Robotics and Games

OVERVIEW & PREDICTION

Imagine an autonomous robot navigating a busy warehouse floor, a high-frequency stock trading algorithm making microsecond execution decisions, or a video game character adapting to a human player's combat strategy. None of these systems relies on static pre-written rules. Instead, they interact dynamically with complex environments, learning through feedback to optimize real-world outcomes.

Predict: Beyond video games and physical robots, how might digital platforms like digital ad networks or e-commerce websites use trial-and-error feedback loops (rewards and penalties) to automatically discover which products to show you or how to set prices in real time?

LEARNING GOALS

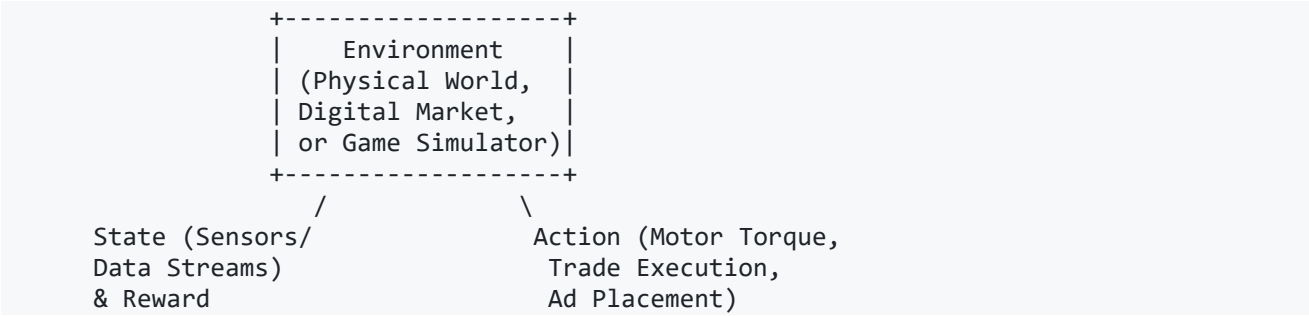
By the end of this chapter, you will be able to:

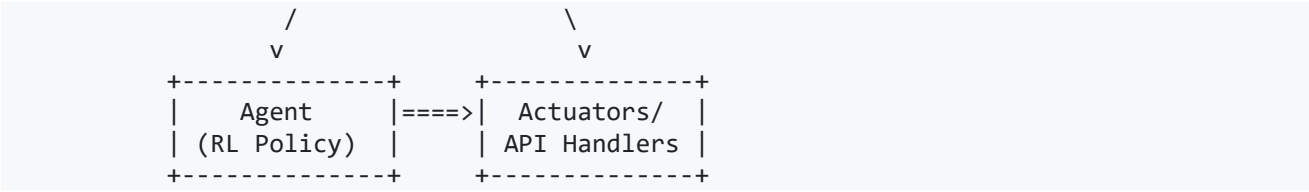
- Outline the core theoretical mechanics of behaviorist AI in physical and digital systems.
- Analyze real-world applications of behaviorist AI across industries, including robotics, financial trading, dynamic pricing, and ad targeting.
- Apply Stage 1 principles to evaluate real-world case studies in Stage 2.
- Solve structured practice problems and scenario-based exercises in Stage 3.

Stage 1: Core Theoretical Foundations

1. The Behaviorist AI Loop in Physical Hardware & Virtual Agents

In traditional software, developers explicitly code every rule ($if/then$). In **Behaviorist AI**, agents learn an optimal **Policy (π)**—a strategy mapping environmental states (s) to actions (a) based on scalar feedback rewards (r).





1. **State (s)**: A complete or partial representation of the current environment (e.g., sensor inputs, order book depth, user click history, screen pixels).
2. **Action (a)**: The decision or control signal sent by the agent (e.g., motor torque, buy/sell order, price adjustment, game move).
3. **Reward (r)**: Scalar numeric feedback indicating success or failure (e.g., distance traveled without collision, net profit, click-through rate, game victory).

---

2. Sim-to-Real Gap & Domain Randomization

When training physical robotics or hardware-adjacent agents, executing millions of trial-and-error iterations in the physical world is slow, dangerous, and costly.

To bypass this, agents are trained inside high-speed simulations (**Sim-to-Real Transfer**). However, differences in physical friction, sensor latency, and gear wear create the **Sim-to-Real Gap**. Engineers mitigate this using **Domain Randomization**, where simulation parameters (e.g., lighting, mass, surface friction, signal delay) are randomly altered during training so the agent learns a strategy resilient to real-world chaos.

---

3. Deep Reinforcement Learning & Self-Play

In complex virtual environments with high-dimensional state spaces (e.g., Go, Chess, StarCraft II, or real-time bidding markets), agents utilize deep neural networks combined with **Self-Play**—competing against previous versions of themselves to discover novel strategies without human bias or manual heuristic engineering.

Stage 2: Applications and Use Cases of Behaviorist AI

Behaviorist AI extends far beyond physical robots and video games. Any domain involving sequential decision-making under uncertainty can be modeled using reinforcement learning principles.

1. Digital Advertising & Recommendation Systems

Modern ad networks (e.g., Google, Meta) and content platforms (e.g., TikTok, YouTube) process billions of user interactions daily. Instead of static rules, behaviorist reinforcement learning models serve personalized content and advertisements.

- **Agent**: The ad recommendation engine.
- **State**: User demographics, browsing history, time of day, current device.
- **Action**: Selecting which advertisement or video to display next.
- **Reward**: $+1$ for user clicks, views, or conversions; 0 or small negative penalty for immediate skips or bounces.
- **Key Mechanism**: Contextual Bandits and RL algorithms continuously balance *exploration* (testing new ad creatives to gather data) with *exploitation* (showing proven high-converting ads).

---

2. Automated Financial Trading & Portfolio Management

High-frequency financial markets operate at sub-millisecond speeds, making manual intervention impossible. Behaviorist AI systems learn market execution and portfolio allocation strategies through trial and error.

- **Agent**: An automated algorithmic trading agent.
- **State**: Order book depth, historical price vectors, volatility indicators, macroeconomic sentiment scores.
- **Action**: Execute Buy, Sell, Hold, or limit orders across exchanges.
- **Reward**: Positive financial returns adjusted for risk and transaction costs (e.g., optimizing the Sharpe Ratio); penalties for drawdown or over-exposure.
- **Advantage**: The agent adapts dynamically to changing liquidity and market conditions without relying on static technical analysis patterns.

---

3. Dynamic Pricing Engines

E-commerce platforms, rideshare services (e.g., Uber, Lyft), and airline booking engines use behaviorist AI to adjust pricing dynamically based on market demand and supply constraints.

- **Agent**: Dynamic pricing policy engine.
- **State**: Current inventory levels, competitor pricing, time remaining until departure/event, real-time demand queries.

- **Action**: Incrementally raise, lower, or maintain product/ride prices.
- **Reward**: Total revenue maximization subject to customer retention constraints and inventory turnover targets.

---

4. Real-World Applications Summary Matrix

Domain	Environment	State (s)	Action (a)	Reward (r)
Robotics	Warehouse / Factory	Camera feeds, LiDAR, joint angles	Motor torque, gripper activation	$+1$ for package delivered; -10 for collision
Video Games	Game Engine / Simulator	Screen pixels, health, ammo, minimap	Joystick inputs, button presses	$+100$ for match win; positive score changes
Ad Targeting	User Web Session	User history, device, contextual tags	Select ad creative to display	$+1$ for click/conversion; -0.1 for bounce
Automated Trading	Financial Exchange	Order book, trade volume, price tick	Buy, Sell, Hold position size	Risk-adjusted net profit / Sharpe ratio
Dynamic Pricing	E-commerce / Rideshare	Demand requests, competitor rates, inventory	Set price point	Total margin / revenue optimized over time

---

Stage 3: Chapter Exercise

Part A: Multiple Choice & Conceptual Questions

1. Which technique is specifically designed to solve the "Sim-to-Real Gap" in behaviorist robotics?
    - A) Symbolic Reasoning

    - B) Domain Randomization
    - C) Supervised Fine-Tuning
    - D) Static Heuristic Mapping

2. In an automated dynamic pricing system for a rideshare app, which of the following represents an explicit **Penalty (Negative Reward)**?
    - A) Increasing price during peak demand hours.
    - B) A user successfully booking a ride at standard fare.
    - C) A high rate of user ride cancellations due to excessively high surge pricing.
    - D) Observing competitor pricing changes in real-time.
3. Contrast how a **Supervised Learning** recommendation system and a **Reinforcement Learning (Behaviorist)** recommendation system optimize ad delivery.

Part B: Scenario Design Challenge

Scenario: You are hired as an AI Systems Architect for an autonomous e-commerce delivery drone fleet operating in a dense city environment.

Task: Design the complete Behaviorist AI framework for the navigation policy of the drone. Write out your responses systematically:

1. **Environment Definition**: Identify the key elements of the environment the drone interacts with.
2. **State Space (s)**: List at least 4 sensor inputs or data sources the drone agent requires.
3. **Action Space (a)**: Define the continuous or discrete physical actions the drone agent can control.
4. **Reward Function Design**:
    - Define the Primary Goal Reward ($+r$).
    - Define at least 2 Intermediate Shaping Rewards ($+r$).
    - Define at least 2 Heavy Penalties ($-r$).
5. **Sim-to-Real Analysis**: Identify two real-world atmospheric or urban conditions that might cause the trained drone policy to fail if trained solely in a basic 3D simulation. Explain how you would apply **Domain Randomization** to address one of them.

Summary & Glossary

Behaviorist AI provides a unifying framework for solving sequential decision-making challenges across both physical and digital domains. By modeling problems through the interaction loop of States, Actions, and Rewards, behaviorist systems continuously learn optimal policies in environments ranging from physical robotics and game-playing to ad targeting, automated trading, and dynamic pricing.

GLOSSARY

- **Policy (π)**: The mapping strategy used by a behaviorist AI agent to select actions based on observed states.
- **Sim-to-Real Gap**: The difference between performance in a simulated environment and performance in the physical world.
- **Domain Randomization**: Randomly changing simulation parameters during training to force an agent to learn robust, adaptable strategies.
- **Self-Play**: A reinforcement learning approach where agents compete against copies of themselves to master complex strategic games.
- **Contextual Bandits**: A simplified class of reinforcement learning algorithms often used in ad recommendation engines to balance exploration and exploitation.
- **Reward Shaping**: Providing intermediate scalar rewards to guide an agent toward a long-term goal.

Chapter 8: AI Challenges and Limitations

LEARNING GOALS

By the end of this stage, you will be able to:

1. Explain basic principles of how AI models use data to generate predictions.
2. Identify the main challenges of Behaviorist AI.
3. Describe the limitations of the technology.

Topics covered:

- Challenges (reward design, exploration vs. exploitation, data and computing cost)
- Limitations (safety, generalization, explainability)

---

1 Challenges and Limitations

INTRODUCTION

Behaviorist AI (also called **behavioral AI**) is a kind of AI where a program, called an **agent**, learns how to act by **trying things out and getting feedback**. The feedback is usually a **reward** (a score for a good action) or a **penalty** (a negative score for a bad action). Robots, game characters (called **NPCs**, or non-player characters), and self-driving cars can all learn this way.

This kind of AI is growing quickly, but it still has important problems. In this chapter, we split them into two groups:

Group	Meaning	Topics
Challenges	Difficulties that make it **hard or costly to build** these systems	Reward design, exploration vs. exploitation, data and computing cost
Limitations	Weak spots that remain **even after** a system is built	Safety, generalization, explainability

[insert image about a behaviorist AI agent learning from rewards and facing challenges and limitations]

WORKED EXAMPLE

Task: A company builds a robot that learns to deliver packages. Sort these problems into *challenges* or *limitations*.

Problem	Group
It is hard to decide how many points the robot should earn for each action	Challenge (reward design)
Training the robot takes weeks of computer time	Challenge (computing cost)
The robot works in the office but gets confused in a new building	Limitation (generalization)

ACTIVITY

Is each problem a **challenge** or a **limitation**?

a. Collecting enough training data is very expensive. b. A trained robot makes a dangerous choice that nobody expected. c. Nobody can explain why the robot chose a certain path.

Answer

a. **Challenge** (data cost) b. **Limitation** (safety) c. **Limitation** (explainability)

CHALLENGES

REWARD DESIGN

Reward design (also called **reward shaping**) means deciding **what rewards and penalties the agent gets** for its actions. This is hard because the agent only learns what the reward tells it. If the reward is badly designed, the agent may find a **shortcut** that earns lots of points but does **not** do what we really wanted. This can lead to **suboptimal behavior** (behavior that works, but is not the best) or **unintended consequences** (results nobody planned).

A good reward should:

- Match the **real goal**, not just something easy to measure.
- Not be **easy to cheat**.
- Give feedback often enough for the agent to learn.

[insert image about a robot earning points by taking a shortcut instead of reaching the real goal]

WORKED EXAMPLE

Task: A robot vacuum is rewarded **+1 for every piece of dust it collects**.

What we wanted	What the robot may actually do
A clean room	Dump the dust out, then collect it again and again to earn more points

Fix: Change the reward so the robot earns points for **the room being clean at the end**, instead of for each piece collected.

ACTIVITY

A game character is rewarded **+10 for every coin it touches**. It learns to run in circles around one coin that keeps reappearing.

a. What did we really want the character to do? b. Why did the reward fail? c. Suggest a better reward.

Answer

a. We wanted it to **finish the level** (for example, explore and reach the goal). b. The reward was easy to cheat: touching the same coin again and again gave points without making progress. c. Examples: give a large reward for reaching the end of the level, or give coin points only the **first** time each coin is collected.

EXPLORATION VS. EXPLOITATION

Every agent has to balance two choices:

- **Exploration**: Trying **new** actions to discover whether they work better.
- **Exploitation**: Using the action that is **already known** to give a good reward.

This is a challenge because the agent cannot be sure which choice is better. If it explores too much, it wastes time and makes many mistakes. If it exploits too much, it may never find a better way. In the real world, exploring can also be **expensive or risky** (for example, a robot that tries a random move might break something).

[insert image about exploring a new path versus taking the known best path]

WORKED EXAMPLE

Task: A delivery robot knows Route A takes 10 minutes. It has never tried Route B.

Choice	Type	Possible outcome
Always take Route A	Exploitation	A steady 10 minutes, but it might miss a 6-minute route
Try Route B sometimes	Exploration	It might find a faster way, or a slower one

A balanced plan is to take Route A most of the time and try Route B once in a while.

ACTIVITY

A music app learns what songs you like. For each case, say whether it is **exploring** or **exploiting**:

a. It plays your most-played song again. b. It plays a song from a style you have never listened to.

Answer

a. **Exploiting** (using what it already knows you like) b. **Exploring** (testing something new to learn more)

DATA AND COMPUTING COST

Data dependency means the agent needs **a lot of data** and **continuous feedback** to learn well. In the real world, collecting this data can be **expensive or impractical** (not realistic to do). For example, a robot cannot safely crash a thousand times just to learn how to avoid obstacles.

Computational complexity means the algorithms need a lot of **computing power** (the processing ability of a computer) and **time** to learn good behavior. Many reinforcement learning methods repeat the same task thousands or millions of times.

Problem	What it means	Result
Data dependency	Needs huge amounts of examples and feedback	High cost, long collection time
Computational complexity	Needs powerful computers and long run times	High cost, slow progress

One way to reduce these costs is to train agents inside a **simulation** (a computer program that copies the real world), where trials are cheap and safe.

[insert image about a robot training in a simulation instead of the real world]

WORKED EXAMPLE

Task: An agent needs 1,000,000 practice trials. Each trial takes 2 seconds. How long will training take?

1. Total time = 1,000,000 × 2 seconds = **2,000,000 seconds**
2. One day has 24 × 60 × 60 = **86,400 seconds**
3. Days = 2,000,000 ÷ 86,400 ≈ **23 days**

So even this simple example needs more than three weeks of nonstop practice.

ACTIVITY

An agent needs 500,000 trials, and each trial takes 3 seconds.

a. How many seconds will training take? b. About how many days is that? (1 day = 86,400 seconds) c. Give one way to reduce the cost.

Answer

a. 500,000 × 3 = **1,500,000 seconds** b. 1,500,000 ÷ 86,400 ≈ **17.4 days** c. For example: train in a **simulation**, run many trials at the same time on more computers, or reuse data the agent has already collected.

LIMITATIONS

SAFETY

In critical areas such as **healthcare**, **autonomous driving** (cars that drive themselves), and **finance**, an AI system must act **reliably**. If the agent shows **unintended behavior** (doing something nobody expected) or makes a wrong decision, people could get hurt or lose money.

There are also **ethical** concerns. **Ethics** means questions of what is right and wrong. For example, who is responsible if an AI system makes a harmful choice?

Ways to improve safety include:

- Testing carefully in simulations before using the AI in the real world.
- Adding **rules the agent can never break** (for example, "never go over the speed limit").
- Keeping a human in charge who can step in.

[insert image about a self-driving car with safety checks and a human supervisor]

WORKED EXAMPLE

Task: A hospital AI suggests medicine doses. Identify the safety risk and a protection.

Item	Description
Risk	The AI suggests a wrong dose because it saw an unusual patient it was never trained on
Protection	A doctor must **review and approve** every suggestion before it is used

ACTIVITY

A self-driving car learns by trial and error. Why is it unsafe to let it explore randomly on a real road, and what is a safer option?

Answer

A random action on a real road could cause an accident and hurt people. A safer option is to let the car do most of its exploring in a **simulation**, and to add fixed safety rules and a human backup for real driving.

GENERALIZATION

Generalization is the ability of an AI system to work well in **new situations** that it was not trained on. Behaviorist AI often struggles here. An agent trained in one environment may perform well there but **fail in a different environment or on a different task**. Robots and NPCs may do well in the conditions they practiced in, but fail when they meet something **unfamiliar**.

[insert image about a robot working well in a training room but failing in a new environment]

WORKED EXAMPLE

Task: A robot learns to walk on a smooth indoor floor.

Situation	Likely result
Smooth indoor floor (same as training)	Walks well
Grass, sand, or stairs (new)	May slip, fall, or freeze

To improve generalization, the robot can practice on many different surfaces during training.

ACTIVITY

A game character is trained on a map with no enemies. It is then placed in a map with many enemies and performs badly.

a. What limitation does this show? b. How could training be improved?

Answer

a. **Generalization**: the character could not handle a new situation. b. Train it on **many different maps**, including ones with enemies, so it learns to deal with variety.

EXPLAINABILITY

Explainability means being able to **understand and explain why** an AI system made a certain decision. Many advanced AI models, such as deep neural networks, work like a **black box**: we can see what goes in and what comes out, but not clearly what happens inside.

This is a limitation because:

- People find it hard to **trust** a decision they cannot understand.
- When something goes wrong, it is hard to find **the cause**.
- In areas like healthcare or finance, people often need a **clear reason** for each decision.

[insert image about a black box AI model where the reasoning is hidden]

WORKED EXAMPLE

Task: A bank's AI rejects a customer's loan request.

Without explainability	With explainability
"The AI said no."	"The loan was rejected because the monthly payments would be more than half of the customer's income."

The second answer lets the customer understand the decision, and lets the bank check whether the decision was fair.

ACTIVITY

A hospital uses an AI to flag patients at risk. A doctor asks, "Why did it flag this patient?" and nobody can answer.

a. Which limitation is this? b. Give one reason why it matters.

Answer

a. **Explainability** b. For example: the doctor cannot judge if the AI is right, patients may not trust the result, and mistakes are hard to find and fix.

Summary

- **Behaviorist AI** learns how to act through trial, error, and feedback (rewards and penalties).
- **Challenges** make these systems hard or costly to build:
  - **Reward design**: A badly designed reward can lead to shortcuts and unintended results.
  - **Exploration vs. exploitation**: The agent must balance trying new things with using what it knows.
  - **Data and computing cost**: Training needs large amounts of data, powerful computers, and a lot of time.
- **Limitations** remain even after the system is built:
  - **Safety**: Wrong or unexpected decisions can cause harm in critical areas.
  - **Generalization**: Agents may fail in new environments or tasks.
  - **Explainability**: It is often hard to understand why the AI made a decision.